

**TRƯỜNG ĐẠI HỌC QUỐC TẾ HỒNG BÀNG
KHOA CÔNG NGHỆ THÔNG TIN**



BÁO CÁO ĐỒ ÁN MÔN HỌC

**XÂY DỰNG ỨNG DỤNG WEB BÁN TRANG SỨC
THANH TÂM NGỌC**

Giảng viên hướng dẫn: ThS. Hoàng Ngọc Long

Sinh viên thực hiện: Huỳnh Bích Trân

Mã số sinh viên: 2411110012

TP. Hồ Chí Minh, 2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án môn học Web App Development, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Hoàng Ngọc Long.

Cảm ơn thầy đã dành nhiều thời gian, tâm huyết để tận tình hướng dẫn, chỉ bảo và truyền đạt những kiến thức quý báu giúp em hoàn thành tốt website Thanh Tâm Ngọc. Những góp ý thực tế của thầy không chỉ giúp em hoàn thiện đồ án lần này mà còn là hành trang kiến thức vững chắc cho em tiếp tục học tập và làm việc sau này.

Mặc dù đã nỗ lực hết mình để hoàn thiện đề tài một cách tốt nhất, nhưng do còn hạn chế về kinh nghiệm thực tế nên đồ án chắc chắn không tránh khỏi những thiếu sót. Kính mong thầy xem xét và đóng góp thêm ý kiến để em có thể rút kinh nghiệm và hoàn thiện kỹ năng lập trình của mình hơn.

Em xin kính chúc thầy thật nhiều sức khỏe và thành công. Em xin chân thành cảm ơn thầy!

TP.HCM, Tháng 6 Năm 2026
Sinh viên thực hiện

Huỳnh Bích Trân

TRANG CAM KẾT

Em tên là Huỳnh Bích Trân, sinh viên lớp CNTT, trường Đại học Quốc tế Hồng Bàng.

Em xin cam kết báo cáo đồ án "Xây dựng ứng dụng web bán trang sức Thanh Tâm Ngọc" được hoàn thành dựa trên các kết quả nghiên cứu và thực hiện của cá nhân em dưới sự hướng dẫn của thầy Hoàng Ngọc Long.

Em cam đoan rằng các kết quả nghiên cứu này là trung thực, chưa từng được sử dụng cho bất cứ báo cáo nào khác và không sao chép từ các công trình nghiên cứu khác. Các tài liệu tham khảo đều đã được em trích dẫn đầy đủ trong báo cáo.

Em xin chân thành cảm ơn!

TP.HCM, Tháng 6 Năm 2026
Huỳnh Bích Trân

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP.HCM, Ngày... Tháng 6 Năm 2026

Chữ ký giảng viên

DANH MỤC BIỂU ĐỒ HÌNH VẼ

- Hình 1.1. Cấu trúc các Controller xử lý nghiệp vụ Back-end của hệ thống.
- Hình 1.2. Cấu trúc lưu trữ dữ liệu của hệ thống.
- Hình 1.3. Mã nguồn sử dụng Fetch API để tải dữ liệu bất đồng bộ.
- Hình 2.1. Thống kê nhu cầu mua sắm trực tuyến tại Việt Nam)
- Hình 2.2. Biểu đồ luồng dữ liệu mức đỉnh
- Hình 2.3. Mô hình thực thể kết hợp (ERD) của hệ thống
- Hình 3.1. Giao diện biểu mẫu Đăng ký tài khoản dành cho khách hàng
- Hình 3.2. Giao diện Đăng nhập hệ thống Thanh Tâm Ngọc
- Hình 3.3. Giao diện yêu cầu nhập Email để khôi phục mật khẩu
- Hình 3.4. Duyệt các sản phẩm thuộc danh mục Mặt dây chuyền
- Hình 3.5. Kết quả tra cứu sản phẩm với từ khóa "Chuỗi"
- Hình 3.6. Sử dụng Dropdown để lọc các sản phẩm theo phân khúc giá
- Hình 3.7. Nút điều hướng phân trang ở cuối danh sách sản phẩm
- Hình 3.8. Giao diện hộp thoại xem thông tin chi tiết của Nhân Phi Thúy
- Hình 3.9. Thông báo trực quan khi khách hàng thêm sản phẩm vào giỏ
- Hình 3.10. Giao diện quản lý chi tiết các sản phẩm trong Giỏ hàng
- Hình 3.11. Cảnh báo khi người dùng nhập thiếu thông tin giao hàng
- Hình 3.12. Giao diện cổng thanh toán tích hợp mã VietQR trực tuyến
- Hình 3.13. Thông báo xác nhận đặt hàng thành công và kết thúc quy trình
- Hình 3.14. Giao diện quản lý thông tin và cập nhật hồ sơ cá nhân
- Hình 3.15. Giao diện theo dõi danh sách lịch sử đơn hàng của khách hàng
- Hình 3.16. Giao diện yêu cầu Đăng nhập bắt buộc tại trang Quản trị
- Hình 3.17. Giao diện tổng quan danh sách Quản lý kho Sản phẩm
- Hình 3.18. Quản trị viên sử dụng thanh tìm kiếm để lọc nhanh sản phẩm
- Hình 3.19. Giao diện danh sách Đơn hàng và thao tác cập nhật Trạng thái
- Hình 3.20. Giao diện tổng quan danh sách Khách hàng hệ thống
- Hình 3.21. Hộp thoại xác nhận trước khi thay đổi trạng thái tài khoản
- Hình 3.22. Giao diện hiển thị sau khi tài khoản đã bị khóa thành công
- Hình 3.23. Giao diện Báo cáo tổng quan (Dashboard) của hệ thống
- Hình 3.24. Hiệu ứng Tooltip hiển thị số liệu chi tiết khi rê chuột vào biểu đồ
- Hình 3.25. Tính năng ẩn/hiện cột dữ liệu động trên biểu đồ Chart.js
- Hình 3.26. Danh sách yêu cầu thiết kế Backend REST API của hệ thống
- Hình 3.27. Mã nguồn cấu hình xác thực và phân quyền (Authorization)
- Hình 3.28. Client gọi Fetch API đến endpoint /api/products để nạp dữ liệu
- Hình 3.29. Mã nguồn C# xử lý truy vấn động tìm kiếm sản phẩm
- Hình 3.30. Mã nguồn kiểm tra và xử lý thêm sản phẩm vào Giỏ hàng
- Hình 3.31. Mã nguồn xử lý tạo lập hóa đơn và chi tiết hóa đơn
- Hình 3.32. Logic xử lý hiển thị cổng thanh toán quét mã QR
- Hình 3.33. Câu lệnh SQL nâng cao xử lý thống kê top sản phẩm
- Hình 3.34. Mã lệnh T-SQL tạo lập cơ sở dữ liệu và các bảng cốt lõi
- Hình 3.35. Cấu trúc các tệp tin lưu trữ mã lệnh SQL Server của dự án

DANH MỤC BẢNG BIỂU

- Bảng 2.1. Cấu trúc bảng USERS (Quản lý tài khoản)
- Bảng 2.2. Cấu trúc bảng CATEGORIES (Quản lý danh mục)
- Bảng 2.3. Cấu trúc bảng PRODUCTS (Quản lý sản phẩm)
- Bảng 2.4. Cấu trúc bảng ORDERS (Quản lý hóa đơn)
- Bảng 2.5. Cấu trúc bảng ORDERDETAILS (Chi tiết hóa đơn)

MỤC LỤC

LỜI CẢM ƠN	2
TRANG CAM KẾT	3
NHẬN XÉT CỦA GIẢNG VIÊN	4
DANH MỤC BIỂU ĐỒ HÌNH VẼ	5
DANH MỤC BẢNG BIỂU	6
LỜI MỞ ĐẦU	8
CHƯƠNG 1.	11
TỔNG QUAN VỀ CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	11
1.1. Tổng quan về thương mại điện tử và website bán hàng	11
1.1.1. Tính cấp thiết của đề tài	11
1.1.2. Lợi ích mang đến cho khách hàng và doanh nghiệp	12
1.1.3. Lợi ích mang đến cho bản thân	12
1.2. GIỚI THIỆU CÔNG NGHỆ SẼ SỬ DỤNG	12
1.2.1. Nền tảng ASP.NET Core Web API (Back-end)	12
1.2.2. Hệ quản trị CSDL SQL Server và Entity Framework	13
1.2.3. Bảo mật với JSON Web Token (JWT)	15
1.2.4. Công nghệ Front-end và Giao tiếp Bất đồng bộ (Fetch API)	15
CHƯƠNG 2. PHÂN TÍCH - THIẾT KẾ HỆ THỐNG	17
2.1. KHẢO SÁT HIỆN TRẠNG	17
2.1.1. Khảo sát hiện trạng về thói quen mua sắm trang sức	17
2.1.2. Hiện trạng kinh tế xã hội và nhu cầu mua sắm trực tuyến	17
2.1.3. Quy trình nghiệp vụ	18
2.1.4. Yêu cầu của website	20
2.2. PHÂN TÍCH – THIẾT KẾ HỆ THỐNG	20
2.2.1. Biểu đồ luồng dữ liệu DFD	20
2.2.2. Mô hình thực thể kết hợp (ERD)	22
2.2.3. Thiết kế cơ sở dữ liệu	22
CHƯƠNG 3. TRIỂN KHAI ỨNG DỤNG	27
3.1. CHỨC NĂNG ĐĂNG KÝ VÀ ĐĂNG NHẬP TÀI KHOẢN (AUTHENTICATION)	27
3.2. HIỂN THỊ DANH MỤC, TÌM KIẾM VÀ LỌC SẢN PHẨM	33
3.3. CHỨC NĂNG XEM CHI TIẾT SẢN PHẨM	37
3.4. CHỨC NĂNG GIỎ HÀNG (SHOPPING CART)	40
3.5. CHỨC NĂNG THANH TOÁN VÀ ĐẶT HÀNG (CHECKOUT)	43
3.6. QUẢN LÝ HỒ SƠ CÁ NHÂN VÀ LỊCH SỬ MUA HÀNG	47
3.7. TỐI ƯU TRẢI NGHIỆM NGƯỜI DÙNG (UX/UI)	50
3.8. PHÂN HỆ QUẢN TRỊ VIÊN (ADMIN DASHBOARD)	51
3.8.1. Chức năng Đăng nhập quản trị	51
3.8.2. Chức năng Quản lý Sản phẩm	53
3.8.3. Chức năng Quản lý Đơn hàng	56
3.8.4. Chức năng Quản lý Người dùng	58
3.8.5. Chức năng Báo cáo Thống kê Tổng quan (Dashboard)	62
3.9. XÂY DỰNG MÁY CHỦ BACK-END (REST API) VÀ CƠ SỞ DỮ LIỆU	66
CHƯƠNG 4: TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN	75
4.1. ĐÁNH GIÁ KẾT QUẢ ĐẠT ĐƯỢC	75
4.2. HƯỚNG PHÁT TRIỂN	76
TÀI LIỆU THAM KHẢO	77

LỜI MỞ ĐẦU

• *Tính cấp thiết của đề tài*

Từ ngàn xưa, trang sức và đá quý không chỉ là những món đồ điểm tô thêm vẻ đẹp cho con người mà còn mang trong mình những giá trị nghệ thuật, văn hóa và phong thủy sâu sắc. Việc sở hữu và trân quý những món trang sức ngọc ngà thể hiện sự tinh tế và đẳng cấp của người sử dụng. Tuy nhiên, trong thời đại công nghệ số hiện nay, thói quen mua sắm của người tiêu dùng đã có sự thay đổi mạnh mẽ. Thay vì phải đến tận cửa hàng truyền thống để lựa chọn, khách hàng ngày càng mong muốn có một không gian mua sắm trực tuyến tiện lợi, an toàn và minh bạch về thông tin.

Việc xây dựng một hệ thống website chuyên nghiệp để giới thiệu và kinh doanh trang sức không chỉ giúp cửa hàng mở rộng thị trường, tiếp cận được nhiều khách hàng tiềm năng mà còn nâng cao uy tín thương hiệu. Từ những nhu cầu thực tiễn đó, em quyết định chọn đề tài "Xây dựng ứng dụng web bán trang sức Thanh Tâm Ngọc" làm đề án môn học với mong muốn áp dụng những kiến thức công nghệ thông tin đã học vào việc giải quyết một bài toán kinh doanh thực tế.

• *Nhận định chung*

Hiện nay, thương mại điện tử đang bùng nổ và trở thành xu hướng tất yếu của mọi ngành hàng. Đối với mặt hàng đặc thù có giá trị cao như trang sức, ngọc đá, khách hàng rất cần sự tin tưởng vào hình ảnh, thông số kỹ thuật (chất liệu, kích thước, màu sắc) và các chính sách bảo hành. Rất nhiều cơ sở kinh doanh trang sức nhỏ lẻ hiện nay vẫn chủ yếu bán hàng qua các trang mạng xã hội (Facebook, Zalo), dẫn đến việc quản lý thông tin sản phẩm rời rạc, khách hàng khó tra cứu, và không có một quy trình đặt hàng, thanh toán tự động chuyên nghiệp.

Việc áp dụng các nền tảng công nghệ web hiện đại để xây dựng một cửa hàng trực tuyến là giải pháp tối ưu nhất. Website không chỉ là bộ mặt của cửa hàng trên môi trường Internet mà còn là công cụ đắc lực hỗ trợ quản lý bán hàng, giúp doanh nghiệp tiết kiệm chi phí vận hành và mang lại trải nghiệm mua sắm tiện nghi nhất cho người dùng.

• *Các vấn đề đặt ra*

- Quá trình quản lý và kinh doanh mặt hàng trang sức theo phương thức truyền thống hoặc qua mạng xã hội đang gặp phải những hạn chế sau:
- Khó khăn trong việc quản lý số lượng lớn sản phẩm với nhiều thuộc tính khác nhau (mẫu mã, giá cả, tồn kho).
- Khách hàng mất nhiều thời gian để tìm kiếm, lọc sản phẩm theo nhu cầu và so sánh giá cả.
- Quy trình tiếp nhận đơn hàng, tính toán tổng giá trị và xử lý thanh toán thực hiện thủ công, dễ xảy ra sai sót.
- Thiếu hệ thống lưu trữ lịch sử mua hàng để chăm sóc khách hàng và thiếu các báo cáo thống kê doanh thu chính xác cho chủ cửa hàng.

• *Mục tiêu của ứng dụng*

- Dựa trên những vấn đề đã đặt ra, ứng dụng web bán trang sức Thanh Tâm Ngọc được xây dựng với các mục tiêu chính:
- Tạo ra một không gian mua sắm trực tuyến trực quan, đẹp mắt và dễ sử dụng cho khách hàng.
- Tự động hóa các quy trình bán hàng, từ khâu duyệt sản phẩm, thêm vào giỏ hàng đến thanh toán trực tuyến.
- Cung cấp công cụ quản trị mạnh mẽ giúp chủ cửa hàng dễ dàng kiểm soát sản phẩm, đơn hàng và khách hàng.
- Xây dựng hệ thống ổn định, bảo mật, tối ưu hóa trải nghiệm người dùng (loading state, pull-to-refresh, animation).

• *Phạm vi dự án*

Hệ thống được thiết kế và phát triển bao gồm 3 phân hệ chính theo tài liệu mô tả:

- **Web Client (Dành cho khách hàng):** Hỗ trợ đăng ký/dăng nhập, duyệt danh mục và tìm kiếm sản phẩm, xem chi tiết sản phẩm, quản lý giỏ hàng, thực hiện thanh toán trực tuyến và tra cứu lịch sử đơn hàng.
- **Admin (Dành cho quản trị viên):** Cung cấp các chức năng quản lý sản phẩm (thêm, sửa, xóa, tồn kho), quản lý đơn hàng (cập nhật trạng thái), quản lý người dùng và xuất báo cáo thống kê doanh thu.
- **Backend (REST API):** Xây dựng nền tảng xử lý dữ liệu trung tâm bao gồm phân quyền (Auth & Roles), API quản lý sản phẩm (CRUD, phân trang), API giỏ hàng, API đơn hàng, tích hợp API thanh toán trực tuyến và API báo cáo.

CHƯƠNG 1.

TỔNG QUAN VỀ CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1. Tổng quan về thương mại điện tử và website bán hàng.

1.1.1. Tính cấp thiết của đề tài

Cùng với sự phát triển không ngừng của nền kinh tế, nhu cầu làm đẹp và khẳng định giá trị bản thân thông qua các món trang sức, đá quý (như Phi Thúy, Ruby, Lam Ngọc, Băng Chủng) của con người ngày càng tăng cao. Trang sức không chỉ mang giá trị thẩm mỹ mà còn mang ý nghĩa phong thủy và lưu trữ tài sản. Tuy nhiên, việc mua bán trang sức theo phương thức truyền thống đòi hỏi khách hàng phải đến tận cửa hàng, mất nhiều thời gian quan sát, lựa chọn và so sánh giá cả.

Hơn nữa, trong thời đại công nghệ số 4.0, người tiêu dùng có xu hướng mua sắm trực tuyến. Đối với các mặt hàng có giá trị cao, khách hàng cực kỳ khắt khe về độ tin cậy, thông tin mô tả chi tiết, hình ảnh chân thực và giấy tờ kiểm định rõ ràng. Nhiều cơ sở kinh doanh trang sức hiện tại vẫn quản lý thủ công hoặc bán hàng qua mạng xã hội một cách rời rạc, dẫn đến khó khăn trong khâu kiểm soát tồn kho, thất thoát dữ liệu đơn hàng và không tạo được sự chuyên nghiệp trong mắt khách hàng.

Nắm bắt được những thách thức đó, ứng dụng web "Thanh Tâm Ngọc" ra đời nhằm tự động hóa quy trình kinh doanh trang sức. Website cung cấp một giải pháp phần mềm hoàn chỉnh giúp số hóa toàn bộ khâu quản lý danh mục sản phẩm, tìm kiếm, đặt hàng và theo dõi doanh thu một cách chính xác, minh bạch.

1.1.2. Lợi ích mang đến cho khách hàng và doanh nghiệp

Đối với khách hàng, website mang lại trải nghiệm mua sắm tiện lợi và an toàn. Chỉ với vài cú click chuột, khách hàng có thể dễ dàng tra cứu, lọc sản phẩm theo mức giá (dưới 5 triệu, trên 10 triệu...), theo loại ngọc, xem chi tiết giấy kiểm định và thêm vào giỏ hàng. Điều này giúp tiết kiệm thời gian đi lại mà vẫn đảm bảo chọn được sản phẩm ưng ý.

Đối với cửa hàng Thanh Tâm Ngọc, hệ thống là một trợ thủ đắc lực giúp quản trị viên (Admin) dễ dàng thêm, sửa, xóa sản phẩm, theo dõi tình trạng tồn kho, phê duyệt đơn hàng và thống kê doanh thu theo thời gian thực. Giao diện báo cáo (Dashboard) trực quan giúp chủ cửa hàng nắm bắt nhanh chóng top các sản phẩm bán chạy, từ đó đưa ra chiến lược kinh doanh hợp lý.

1.1.3. Lợi ích mang đến cho bản thân

Quá trình thực hiện đồ án này là cơ hội quý báu để bản thân áp dụng những kiến thức lý thuyết chuyên ngành Công nghệ thông tin vào một bài toán thực tế. Qua việc tự tay phân tích, thiết kế cơ sở dữ liệu, xây dựng các luồng xử lý API và lập trình giao diện, em đã hoàn thiện được tư duy tư duy logic và nâng cao kỹ năng sử dụng các công nghệ hiện đại như ASP.NET Core, C# và SQL Server. Đây là bước đệm vững chắc, tạo tiền đề kinh nghiệm thực tiễn quan trọng cho công việc của một lập trình viên phần mềm sau khi tốt nghiệp.

1.2. GIỚI THIỆU CÔNG NGHỆ SẼ SỬ DỤNG

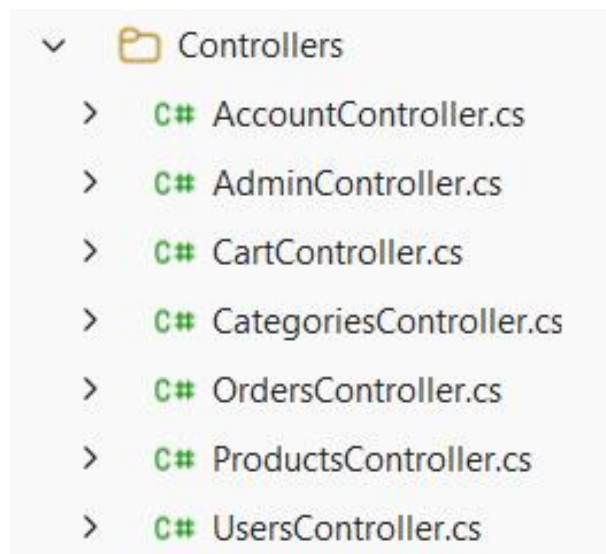
1.2.1. Nền tảng ASP.NET Core Web API (Back-end)

ASP.NET Core là một framework mã nguồn mở, đa nền tảng và có hiệu suất cực kỳ cao do Microsoft phát triển. Thay vì viết code xử lý trộn lẫn

với giao diện, hệ thống "Thanh Tâm Ngọc" được xây dựng theo chuẩn RESTful API.

Với kiến trúc này, phía Server chỉ đảm nhận việc xử lý logic nghiệp vụ và trả về dữ liệu thuần dưới định dạng JSON. Cấu trúc mã nguồn được phân chia thành các Controller rõ ràng, chuyên biệt hóa từng nhiệm vụ:

- AccountController: Xử lý đăng ký, đăng nhập, khôi phục mật khẩu.
- ProductsController: Xử lý các nghiệp vụ truy vấn, thêm, sửa, xóa (CRUD) danh sách trang sức.
- OrdersController: Xử lý quy trình đặt hàng, lưu chi tiết giỏ hàng vào cơ sở dữ liệu.
- AdminController: Trích xuất dữ liệu tổng hợp phục vụ việc vẽ biểu đồ thống kê doanh thu.



Hình 1.1. Cấu trúc các Controller xử lý nghiệp vụ Backend của hệ thống.

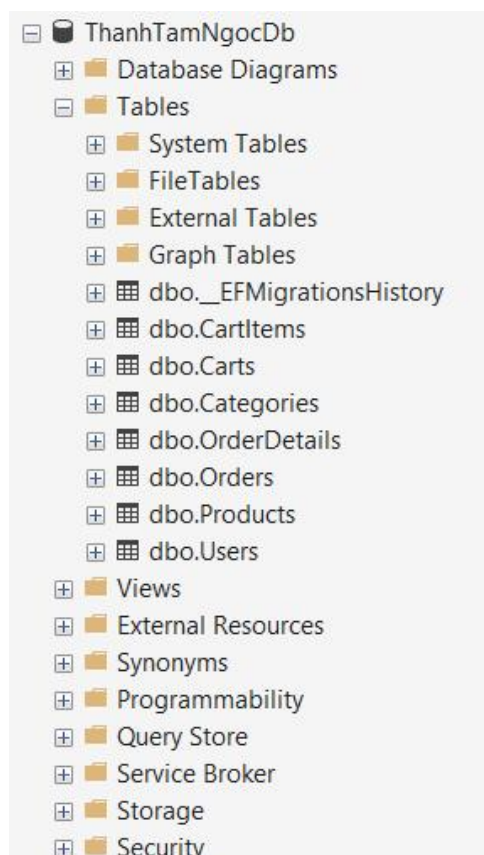
1.2.2. Hệ quản trị CSDL SQL Server và Entity Framework

Mọi dữ liệu cốt lõi của hệ thống được lưu trữ an toàn, tập trung bằng hệ quản trị cơ sở dữ liệu quan hệ Microsoft SQL Server. Dữ liệu được chuẩn hóa

và phân chia thành các bảng có mối quan hệ chặt chẽ với nhau thông qua các Khóa chính (Primary Key) và Khóa ngoại (Foreign Key) như: Bảng Users (Người dùng), Categories (Danh mục), Products (Sản phẩm), Orders (Đơn hàng) và OrderDetails (Chi tiết đơn hàng).

Để hệ thống giao tiếp với cơ sở dữ liệu, đề án sử dụng kết hợp hai kỹ thuật:

- **ADO.NET:** Sử dụng các đối tượng SqlConnection và SqlCommand để thực thi các câu lệnh truy vấn SQL trực tiếp, giúp tối ưu hóa tối đa tốc độ xử lý khi đọc danh sách hàng trăm sản phẩm hoặc lưu trữ hóa đơn.
- **Entity Framework Core:** Công cụ ORM (Object-Relational Mapping) giúp ánh xạ các bảng trong database thành các đối tượng (Model) trong ngôn ngữ C#, hỗ trợ tốt trong việc tự động khởi tạo dữ liệu mẫu (Seed Data) và quản lý giỏ hàng (Cart, CartItem).



Hình 1.2. Cấu trúc lưu trữ dữ liệu của hệ thống.

1.2.3. Bảo mật với JSON Web Token (JWT)

An toàn thông tin là ưu tiên hàng đầu của một website bán hàng. Ứng dụng triển khai cơ chế xác thực không lưu trạng thái (Stateless Authentication) bằng JSON Web Token. Khi khách hàng hoặc quản trị viên đăng nhập thành công, hệ thống không sử dụng Session truyền thống mà tạo ra một chuỗi Token được mã hóa bằng thuật toán HMAC SHA256.

Chuỗi Token này chứa các thông tin định danh (Claim) như ID, Username và Phân quyền (Role). Client sẽ lưu trữ Token và đính kèm vào phần Header (Bearer) của các yêu cầu gửi lên Server tiếp theo. Cách tiếp cận này không chỉ bảo mật cao, chống lại các cuộc tấn công CSRF (Cross-Site Request Forgery) mà còn giúp hệ thống hoạt động cực kỳ mượt mà.

1.2.4. Công nghệ Front-end và Giao tiếp Bất đồng bộ (Fetch API)

Giao diện người dùng (Front-end) được xây dựng hoàn toàn độc lập với Back-end, sử dụng sức mạnh của HTML5, CSS3 và Vanilla JavaScript.

Bố cục website được thiết kế linh hoạt, màu sắc xanh ngọc chủ đạo làm nổi bật tính chất sang trọng của mặt hàng trang sức. Giao diện áp dụng các kỹ thuật như Flexbox và Grid để tự động co giãn, tương thích hiển thị trên nhiều loại màn hình khác nhau.

Fetch API: Thay vì mỗi lần người dùng bấm lọc sản phẩm hay thêm vào giỏ hàng thì trang web phải xoay đều (reload) lại từ đầu, hệ thống sử dụng Fetch API kết hợp với kỹ thuật xử lý bất đồng bộ (Promises/async-await). Giao diện gửi request ngầm lên ASP.NET API, lấy cục dữ liệu JSON về và dùng JavaScript để "vẽ" lại danh sách sản phẩm, cập nhật số lượng giỏ hàng ngay lập tức. Điều này mang lại trải nghiệm thao tác siêu nhanh, mượt mà giống hệt các ứng dụng phần mềm độc lập (Single Page Application).

```
        .catch(err => alert("Không thể kết nối đến máy chủ!"));
    }

    function loadProducts() {
        fetch('/api/products')
        .then(res => res.json())
        .then(data => {
            allProducts = data;
            filteredProducts = data;
            renderProducts();
        })
        .catch(err => console.error(err));
    }

    function renderProducts() {
        const container = document.getElementById('product-list');
    }
```

Hình 1.3. Mã nguồn sử dụng Fetch API để tải dữ liệu bất đồng bộ.

CHƯƠNG 2. PHÂN TÍCH - THIẾT KẾ

HỆ THỐNG

2.1. KHẢO SÁT HIỆN TRẠNG

2.1.1. Khảo sát hiện trạng về thói quen mua sắm trang sức

Cùng với sự phát triển của đời sống, nhu cầu sở hữu các loại trang sức, đá quý thiên nhiên (như Phỉ Thúy, Ngọc Lục Bảo, Ruby, Lam Ngọc, Băng Chủng) không chỉ dừng lại ở việc làm đẹp mà còn mang giá trị phong thủy và tích lũy tài sản. Tuy nhiên, theo khảo sát hành vi người tiêu dùng, phần lớn khách hàng gặp khó khăn khi tìm kiếm sản phẩm chất lượng, đúng giá trị thực qua các kênh truyền thông.

Khách hàng thường mất nhiều thời gian đến trực tiếp các cửa hàng để xem mẫu mã, hoặc theo dõi qua các kênh mạng xã hội nhỏ lẻ với thông tin rời rạc, hình ảnh không rõ nét và quy trình mua bán không được tự động hóa. Hơn nữa, với các mặt hàng giá trị cao, khách hàng đặc biệt quan tâm đến nguồn gốc và giấy tờ kiểm định (Certificate), nhưng không phải nơi nào cũng cung cấp thông tin minh bạch.

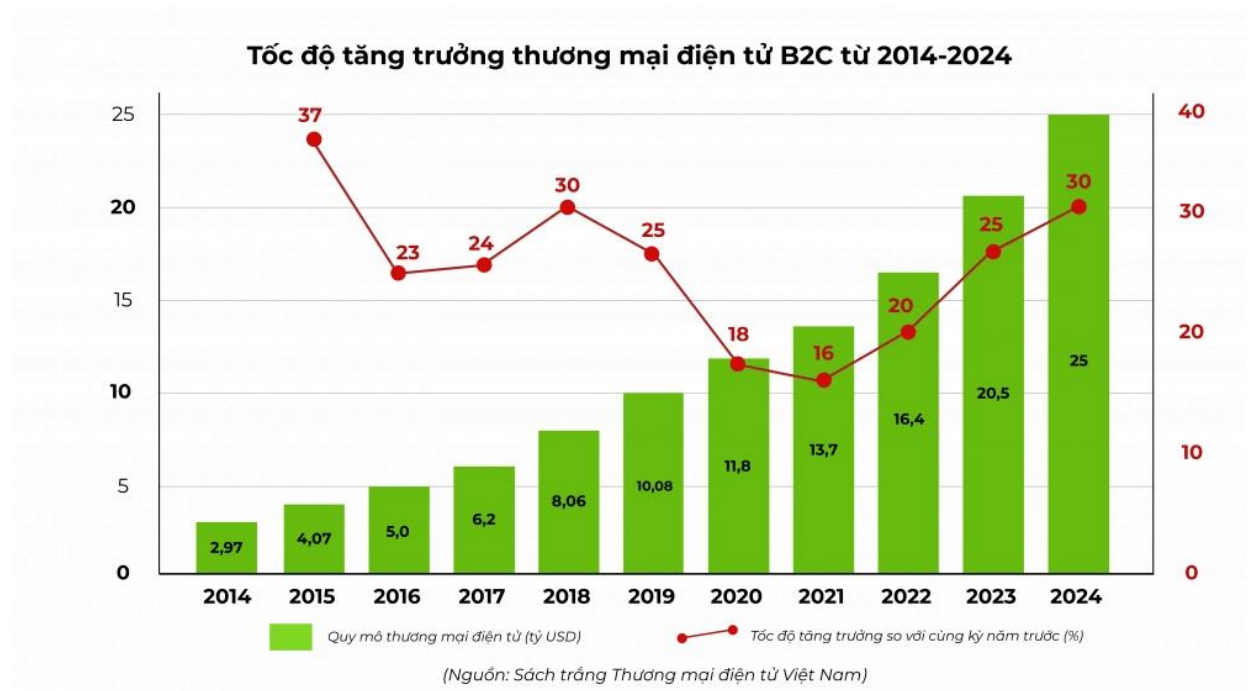
2.1.2. Hiện trạng kinh tế xã hội và nhu cầu mua sắm trực

tuyến

Hiện nay, thương mại điện tử tại Việt Nam đang có những bước tiến vượt bậc, đặc biệt là tại các trung tâm kinh tế lớn như Thành phố Hồ Chí Minh. Tỷ lệ

người dùng sử dụng Internet và các thiết bị di động thông minh để mua sắm ngày càng tăng cao.

Việc ứng dụng công nghệ thông tin vào quản lý và phân phối trang sức không chỉ giúp cửa hàng mở rộng phạm vi tiếp cận khách hàng trên toàn quốc mà còn mang lại trải nghiệm mua sắm tiện lợi, nhanh chóng. Khách hàng có thể dễ dàng xem chi tiết hình ảnh, mức giá, loại ngọc và giấy kiểm định ngay trên website mà không cần phải di chuyển.



Hình 2.1. Thống kê nhu cầu mua sắm trực tuyến tại Việt Nam)

2.1.3. Quy trình nghiệp vụ

Đây là một website thương mại điện tử chứa lượng lớn thông tin về các mặt hàng trang sức đá quý. Quy trình nghiệp vụ được chia theo từng đối tượng người dùng cụ thể:

2.1.3.1. Đối với khách ghé thăm trang web

Khi khách hàng ghé thăm địa chỉ website, giao diện chính sẽ hiện ra với các sản phẩm nổi bật (Featured Products) được hiển thị bắt mắt. Khách hàng có thể

sử dụng thanh menu để duyệt sản phẩm theo các danh mục như: Nhẫn, Vòng tay, Mặt dây chuyền, Ngọc bội. Đồng thời, website cung cấp chức năng lọc thông minh (theo từ khóa, theo mức giá) giúp người dùng tìm được sản phẩm ưng ý nhanh nhất. Khi click vào sản phẩm, hệ thống hiển thị chi tiết hình ảnh, giá cả, tình trạng tồn kho, chất liệu ngọc và mô tả chi tiết.

2.1.3.2. Đối với khách hàng thành viên

Để thực hiện chức năng mua hàng, người dùng cần đăng ký và đăng nhập tài khoản.

- Sau khi đăng nhập, khách hàng có thể thêm các sản phẩm ưng ý vào Giỏ hàng.
- Trong giỏ hàng, khách hàng được quyền điều chỉnh số lượng hoặc xóa sản phẩm.
- Quá trình thanh toán (Checkout) diễn ra nhanh chóng, hệ thống tự động tải lên thông tin nhận hàng từ hồ sơ của thành viên. Người dùng có thể chọn hình thức thanh toán khi nhận hàng (COD) hoặc quét mã QR thanh toán trực tuyến. Sau khi đặt hàng, khách hàng có thể tra cứu lịch sử mua hàng của mình.

2.1.3.3. Đối với người quản trị website (Admin)

Người quản trị có tài khoản riêng biệt để truy cập vào phân hệ Dashboard, thực hiện các chức năng:

- Quản lý chung: Xem các báo cáo thống kê doanh thu, số lượng đơn hàng, người dùng và top 5 sản phẩm bán chạy nhất.
- Quản lý sản phẩm: Xem danh sách, thêm, sửa, xóa sản phẩm, cập nhật số lượng tồn kho.
- Quản lý đơn hàng: Xem chi tiết hóa đơn khách đặt, cập nhật trạng thái đơn hàng (Đang chờ xử lý, Đang giao hàng, Thành công, Đã hủy).

- Quản lý người dùng: Xem thông tin khách hàng, thực hiện khóa hoặc mở khóa tài khoản.

2.1.4. Yêu cầu của website

- Giao diện bắt mắt, sang trọng, màu sắc phù hợp với ngành hàng đá quý. Sử dụng công nghệ Responsive Web Design (RWD) để tương thích hiển thị trên nhiều thiết bị (PC, Laptop, Điện thoại).
- Cung cấp bộ công cụ tìm kiếm và lọc dữ liệu (theo giá, theo danh mục) linh hoạt và xử lý bất đồng bộ (không tải lại trang).
- Hệ thống cơ sở dữ liệu lưu trữ thông tin sản phẩm, khách hàng, đơn hàng chặt chẽ, bảo mật và toàn vẹn.

2.2. PHÂN TÍCH – THIẾT KẾ HỆ THỐNG

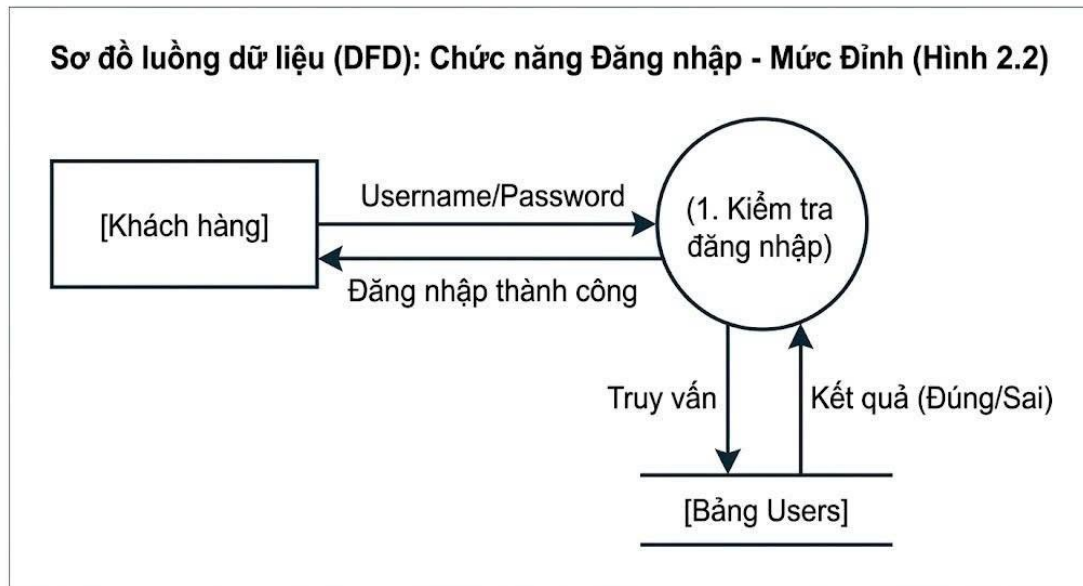
2.2.1. Biểu đồ luồng dữ liệu DFD

2.2.1.1. Định nghĩa

Biểu đồ luồng dữ liệu DFD là một sơ đồ hình học nhằm diễn tả các luồng thông tin và dữ liệu thông qua các chức năng của hệ thống. Tùy theo quy mô mà sự phân cấp được chia thành các mức từ tổng quát đến chi tiết (Mức khung cảnh, Mức đỉnh, Mức dưới đỉnh).

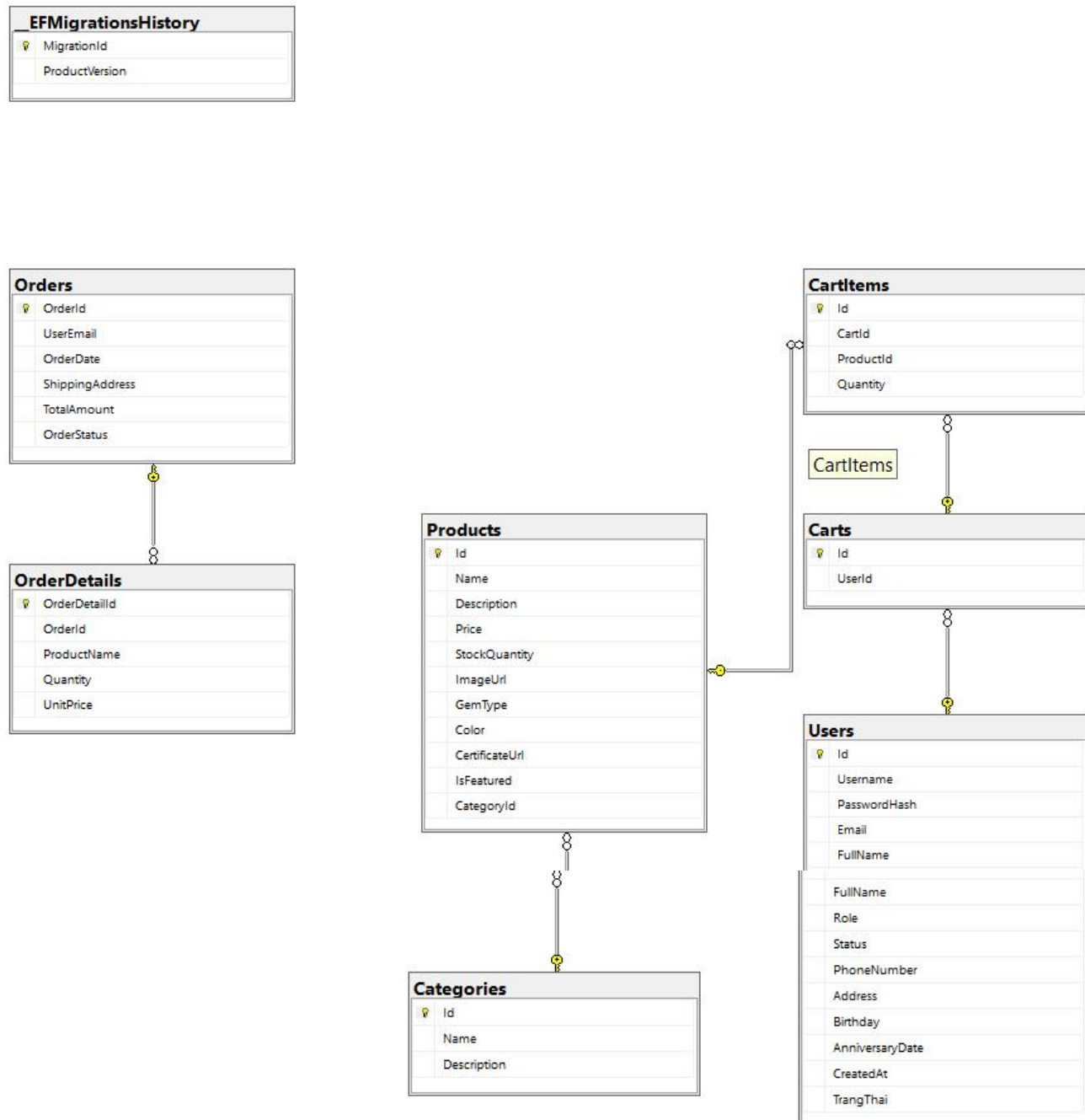
2.2.1.2. Hỗ trợ của DFD

- Xác định yêu cầu của người dùng.
- Lập kế hoạch và minh họa những phương án cho người phân tích thiết kế.
- Làm tài liệu đặc tả yêu cầu hình thức và thiết kế hệ thống.



Hình 2.2. Biểu đồ luồng dữ liệu mức đỉnh

2.2.2. Mô hình thực thể kết hợp (ERD)



Hình 2.3. Mô hình thực thể kết hợp (ERD) của hệ thống

2.2.3. Thiết kế cơ sở dữ liệu

2.2.3.1. Bảng USERS (Quản lý tài khoản)

FieldName	Type	Constraint
Id	Int	Primary key, Auto increment
Username	Nvarchar(50)	Not null
PasswordHash	Nvarchar(max)	Not null
Email	Nvarchar(max)	Not null
FullName	Nvarchar(100)	Not null
Role	Nvarchar(max)	Default = 'Customer'
TrangThai	Int	Default = 1
PhoneNumber	Nvarchar(20)	Nullable
Address	Nvarchar(max)	Nullable

Diễn giải:

- **Id:** Là khóa chính dùng để nhận diện từng tài khoản người dùng trong hệ thống (tự động tăng).
- **Username, PasswordHash:** Tên đăng nhập và mật khẩu (đã được mã hóa) dùng để truy cập hệ thống.
- **Role:** Phân quyền hệ thống, giá trị "Admin" cho quản trị viên và "Customer" cho khách hàng.
- **TrangThai:** Thể hiện trạng thái tài khoản đang hoạt động (1) hoặc bị khóa (0).

2.2.3.2. Bảng CATEGORIES (Quản lý danh mục)

FieldName	Type	Constraint
Id	Int	Primary key, Auto increment
Name	Nvarchar(100)	Not null
Description	Nvarchar(500)	Nullable

Diễn giải:

- **Id:** Là khóa chính của bảng danh mục (Nhẫn, Vòng tay, Mặt dây chuyền...).
- **Name:** Tên hiển thị của danh mục trên giao diện người dùng.

2.2.3.3. Bảng PRODUCTS (Quản lý sản phẩm)

FieldName	Type	Constraint
Id	Int	Primary key, Auto increment
Name	Nvarchar(200)	Not null
CategoryId	Int	Foreign key, Not null
Price	Decimal(18,2)	Not null
StockQuantity	Int	Not null, Default = 0
ImageUrl	Nvarchar(max)	Not null
GemType	Nvarchar(100)	Not null
Description	Nvarchar(max)	Not null

Diễn giải:

- **Id:** Khóa chính của bảng sản phẩm.
- **CategoryId:** Khóa ngoại liên kết với bảng Categories để phân loại sản phẩm.
- **Price, StockQuantity:** Lưu thông tin giá bán và số lượng hàng còn tồn trong kho.
- **ImageUrl:** Đường dẫn lưu trữ hình ảnh minh họa cho sản phẩm.

2.2.3.4. Bảng ORDERS (Quản lý hóa đơn)

FieldName	Type	Constraint
OrderId	Int	Primary key, Auto increment
UserEmail	Varchar(100)	Not null
OrderDate	Datetime	Default = GETDATE()
ShippingAddress	Nvarchar(255)	Not null
TotalAmount	Decimal(18,2)	Not null
OrderStatus	Nvarchar(50)	Default = 'Thành công'

Diễn giải:

- **OrderId:** Khóa chính lưu mã đơn hàng.
- **UserEmail:** Thông tin khách hàng đã thực hiện đặt đơn hàng.
- **ShippingAddress, TotalAmount:** Địa chỉ giao hàng và tổng số tiền thanh toán của hóa đơn.
- **OrderStatus:** Trạng thái hiện tại của đơn hàng.

2.2.3.5. Bảng ORDERDETAILS (Chi tiết hóa đơn)

FieldName	Type	Constraint
OrderDetailId	Int	Primary key, Auto increment
OrderId	Int	Foreign key, Not null
ProductName	Nvarchar(200)	Not null
Quantity	Int	Not null
UnitPrice	Decimal(18,2)	Not null

Diễn giải:

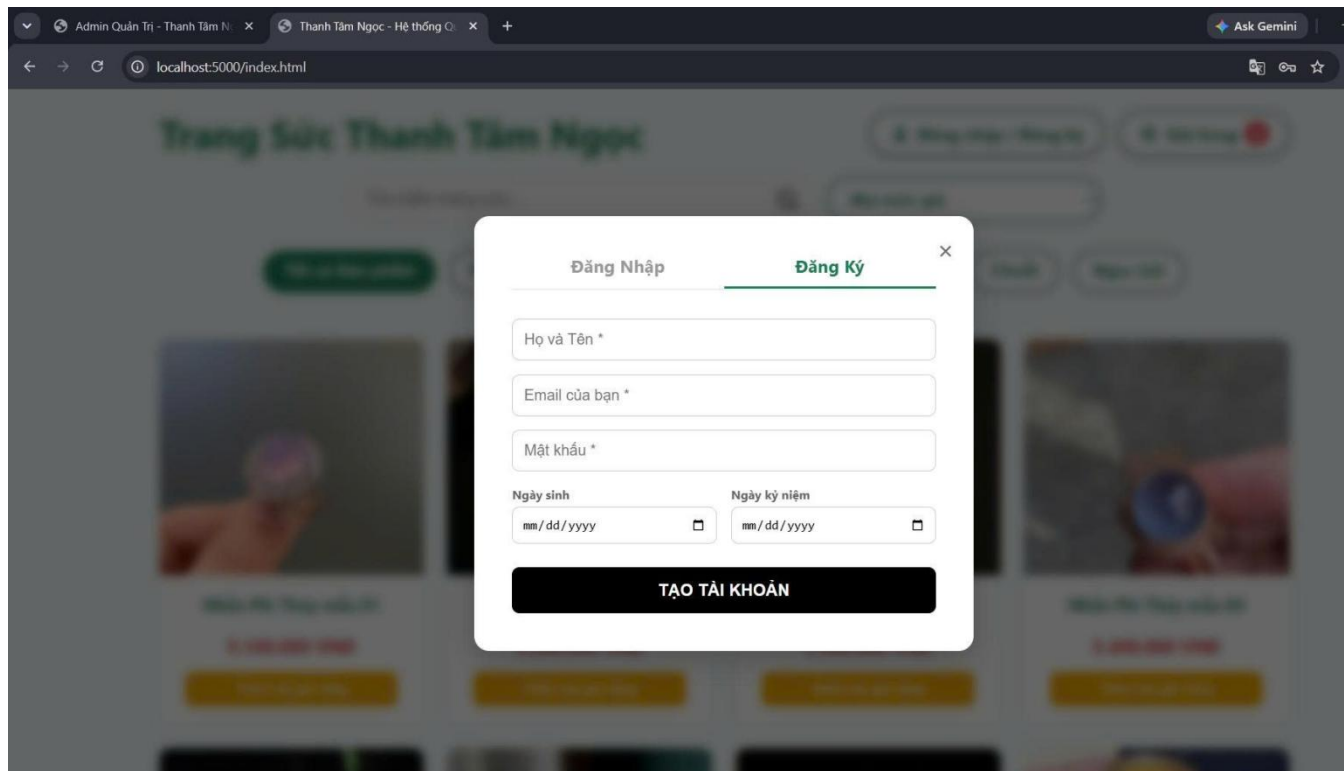
- **OrderDetailId:** Khóa chính của bảng chi tiết.
- **OrderId:** Khóa ngoại liên kết với bảng Orders để biết chi tiết này thuộc về đơn hàng nào. Phân rã dữ liệu giúp một hóa đơn có thể chứa nhiều mặt hàng khác nhau.
- **Quantity, UnitPrice:** Số lượng và đơn giá của sản phẩm tại thời điểm khách hàng đặt mua.

CHƯƠNG 3. TRIỂN KHAI ỨNG DỤNG

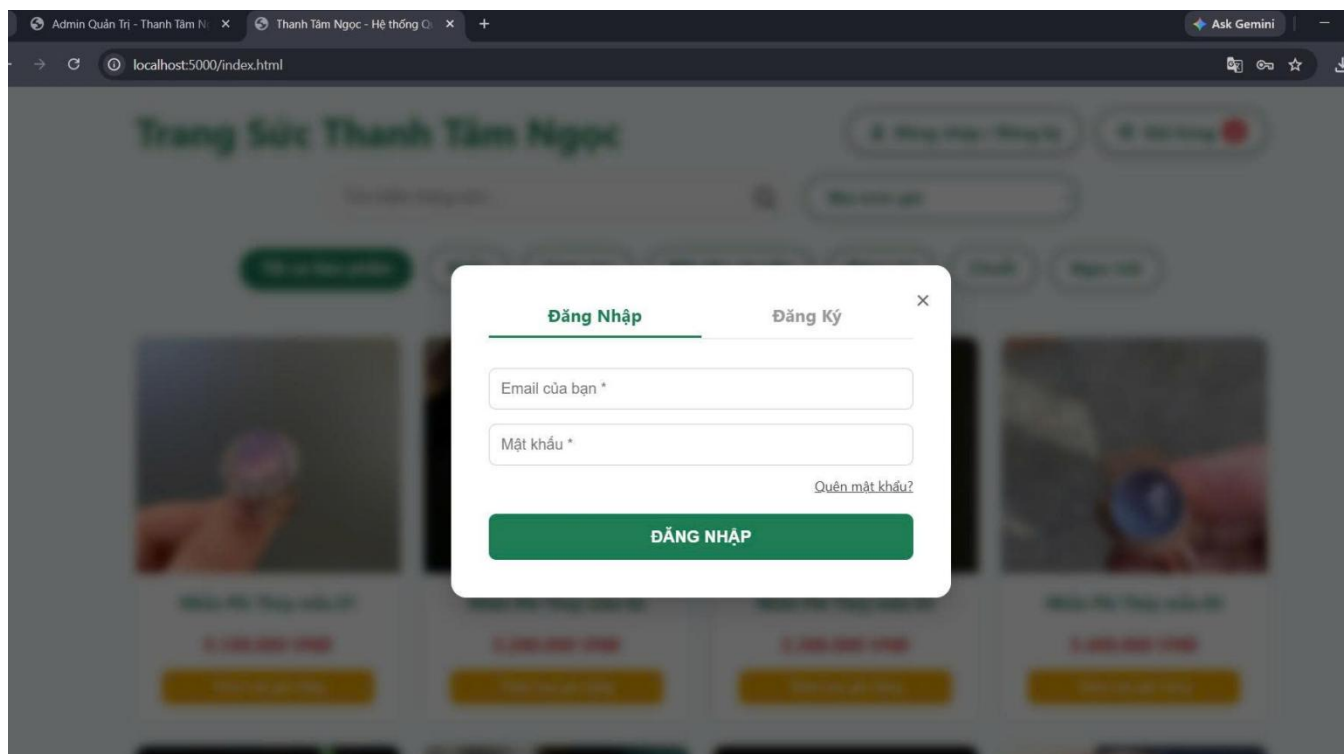
3.1. CHỨC NĂNG ĐĂNG KÝ VÀ ĐĂNG NHẬP TÀI KHOẢN (AUTHENTICATION)

a. Giao diện người dùng (Front-end)

Chức năng Đăng ký và Đăng nhập được thiết kế dưới dạng hộp thoại nổi (Modal Popup) ngay trên trang chủ index.html. Việc áp dụng thiết kế này giúp tối ưu hóa trải nghiệm người dùng (UX), cho phép khách hàng thực hiện thao tác ghi danh hoặc truy cập hệ thống một cách nhanh chóng mà không cần phải chuyển hướng (redirect) hay tải lại toàn bộ trang web.



Hình 3.1. Giao diện biểu mẫu Đăng ký tài khoản dành cho khách hàng



Hình 3.2. Giao diện Đăng nhập hệ thống Thanh Tâm Ngọc

b. Xử lý nghiệp vụ (Back-end)

Dữ liệu từ biểu mẫu phía Client sẽ được đóng gói dưới định dạng JSON và gửi đến Server thông qua Fetch API. Tại phía máy chủ (Back-end), tiến trình này được đảm nhận bởi tệp tin **AccountController.cs** nằm trong thư mục Controllers.

Đối với chức năng Đăng ký, hệ thống sử dụng đối tượng SqlConnection và SqlCommand để kết nối trực tiếp với cơ sở dữ liệu SQL Server. Mã nguồn thực hiện hai bước kiểm tra logic quan trọng:

1. Truy vấn kiểm tra sự tồn tại của Email trong hệ thống để tránh trùng lặp tài khoản.
2. Nếu hợp lệ, tiến hành thực thi câu lệnh INSERT INTO để lưu thông tin khách hàng (Tên, Email, Mật khẩu, Ngày sinh...) vào bảng Users.

Đoạn mã nguồn xử lý đăng ký tài khoản (Trích từ tệp AccountController.cs):

```
[HttpPost("register")]
public IActionResult Register([FromBody] RegisterRequest request)
{
    using (SqlConnection connection = new SqlConnection(_connectionString))
    {
        connection.Open();

        // Bước 1: Kiểm tra Email đã tồn tại hay chưa
        string checkExistQuery = "SELECT COUNT(1) FROM Users WHERE Email = @Email";
        using (SqlCommand checkCmd = new SqlCommand(checkExistQuery, connection))
        {
            checkCmd.Parameters.AddWithValue("@Email", request.Email);
            int exists = Convert.ToInt32(checkCmd.ExecuteScalar());
            if (exists > 0)
            {
                return Conflict("Email already exists.");
            }
        }

        // Bước 2: Thêm người dùng mới vào cơ sở dữ liệu
        string insertQuery = @"
        INSERT INTO Users (FullName, Email, PasswordHash, Birthday, AnniversaryDate, CreatedAt)
        VALUES (@FullName, @Email, @PasswordHash, @Birthday, @AnniversaryDate, GETDATE())";

        using (SqlCommand insertCmd = new SqlCommand(insertQuery, connection))
        {
            insertCmd.Parameters.AddWithValue("@FullName", request.FullName);
            insertCmd.Parameters.AddWithValue("@Email", request.Email);
            insertCmd.Parameters.AddWithValue("@PasswordHash", request.Password);
            // Xử lý giá trị null cho ngày sinh và kỷ niệm
            insertCmd.Parameters.AddWithValue("@Birthday", request.Birthday.HasValue ? (object)request.Birthday.Value : DBNull.Value);

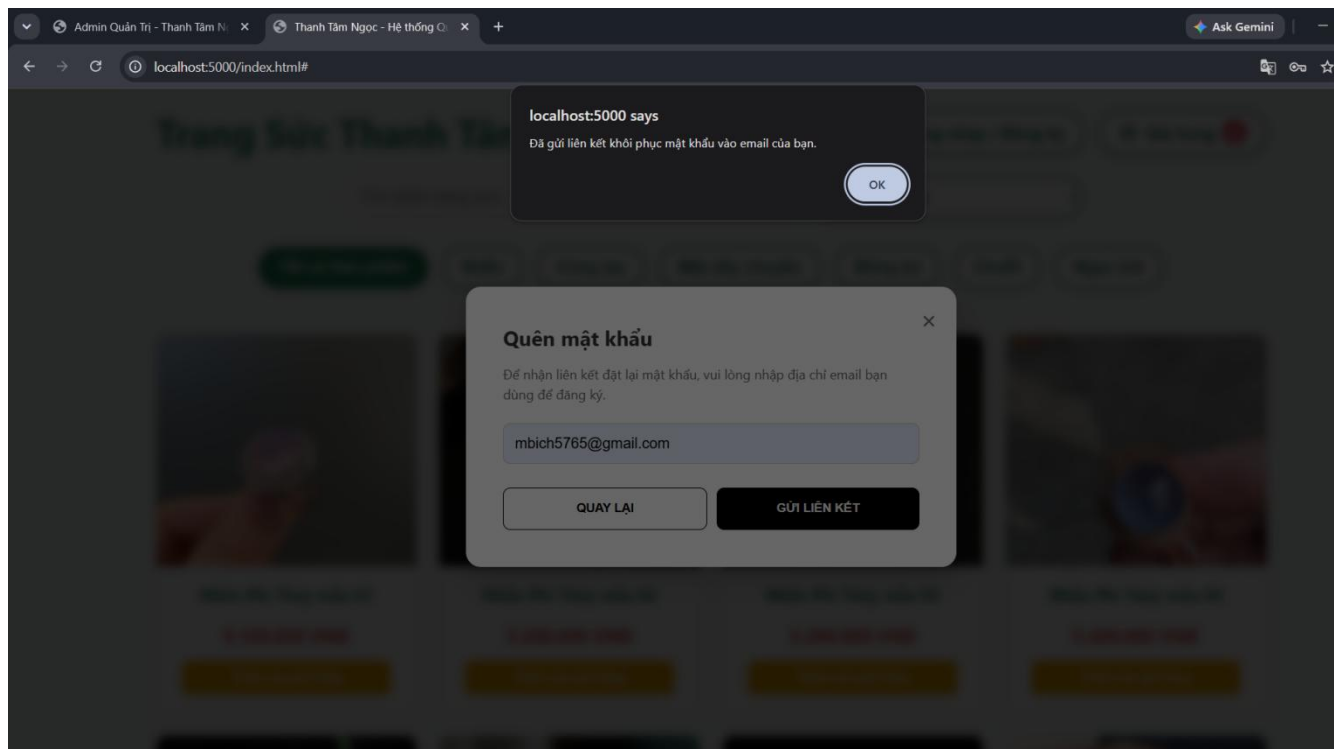
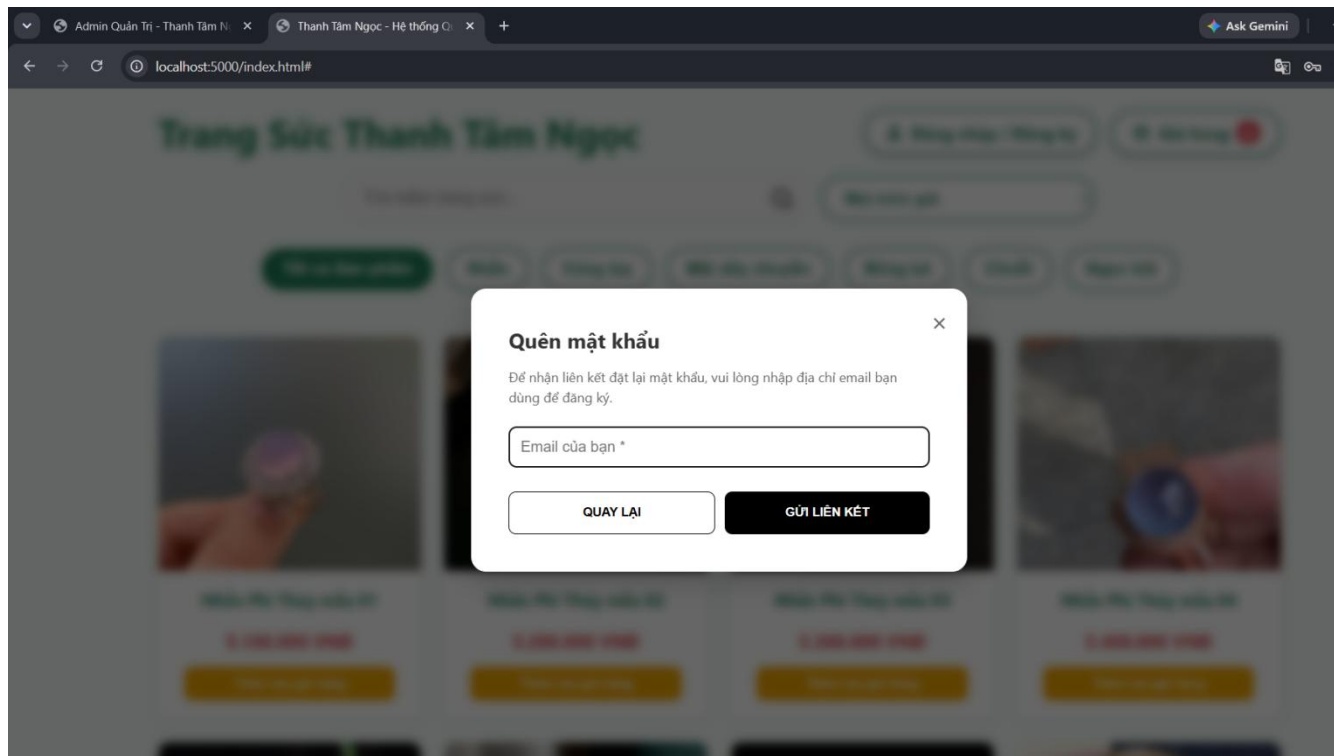
            int rowsAffected = insertCmd.ExecuteNonQuery();
            if (rowsAffected > 0)
            {
                return Ok("Registration successful.");
            }
            else
            {
                return StatusCode(500, "Failed to register user.");
            }
        }
    }
}
```

Đối với chức năng Đăng nhập, API [HttpPost("login")] sẽ đối chiếu thông tin và cấp quyền truy cập vào hệ thống, sau đó cập nhật giao diện người dùng để hiển thị các chức năng mua sắm và quản lý hồ sơ cá nhân.

c. Chức năng Khôi phục mật khẩu (Forgot Password)

Bên cạnh Đăng nhập và Đăng ký, hệ thống còn cung cấp tính năng hỗ trợ khách hàng lấy lại quyền truy cập khi quên mật khẩu.

Về mặt giao diện, khi người dùng nhấn vào "Quên mật khẩu?", một biểu mẫu nhỏ sẽ xuất hiện yêu cầu nhập địa chỉ Email đã dùng để đăng ký.



Hình 3.3. *Giao diện yêu cầu nhập Email để khôi phục mật khẩu*

Về mặt xử lý nghiệp vụ, API [HttpPost("forgotpassword")] sẽ tiếp nhận Email từ phía Client. Hệ thống sử dụng lệnh SELECT COUNT(1) để kiểm tra đối chiếu trong cơ sở dữ liệu. Nếu Email tồn tại trong bảng Users, một thông báo thành công sẽ được trả về (trong thực tế sẽ tích hợp thêm dịch vụ gửi thư SMTP để gửi link đặt lại mật khẩu). Nếu Email chưa từng đăng ký, hệ thống sẽ báo lỗi ngay lập tức để người dùng kiểm tra lại.

Đoạn mã nguồn xử lý đối chiếu Email khôi phục (Trích từ tệp AccountController.cs):

```
[HttpPost("forgotpassword")]
public IActionResult ForgotPassword([FromBody] ForgotPasswordRequest request)
{
    if (string.IsNullOrEmpty(request.Email))
    {
        return BadRequest("Vui lòng nhập email.");
    }

    try
    {
        using (SqlConnection connection = new SqlConnection(_connectionString))
        {
            connection.Open();
            // Kiểm tra xem Email có tồn tại trong hệ thống không
            string query = "SELECT COUNT(1) FROM Users WHERE Email = @Email";
            using (SqlCommand cmd = new SqlCommand(query, connection))
            {
                cmd.Parameters.AddWithValue("@Email", request.Email);
                int exists = Convert.ToInt32(cmd.ExecuteScalar());

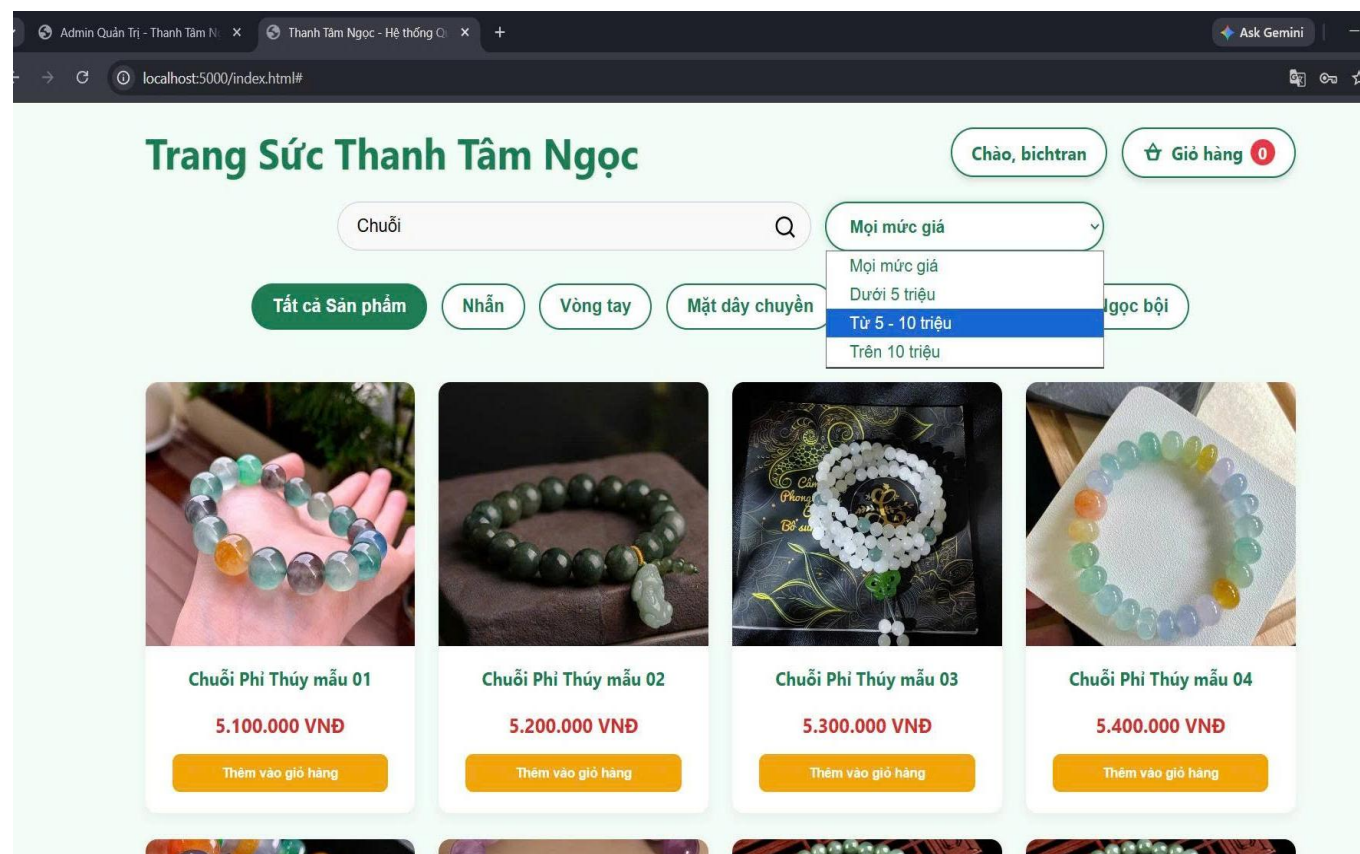
                if (exists > 0)
                {
                    // Giả lập gửi liên kết thành công
                    return Ok("Đã gửi liên kết khôi phục mật khẩu vào email của bạn.");
                }
                else
                {
                    return NotFound("Email này chưa được đăng ký trong hệ thống.");
                }
            }
        }
    }
    catch (Exception ex)
    {
        return StatusCode(500, "Internal server error: " + ex.Message);
    }
}
```

3.2. HIỂN THỊ DANH MỤC, TÌM KIẾM VÀ LỌC SẢN PHẨM

a. Giao diện người dùng (Front-end)

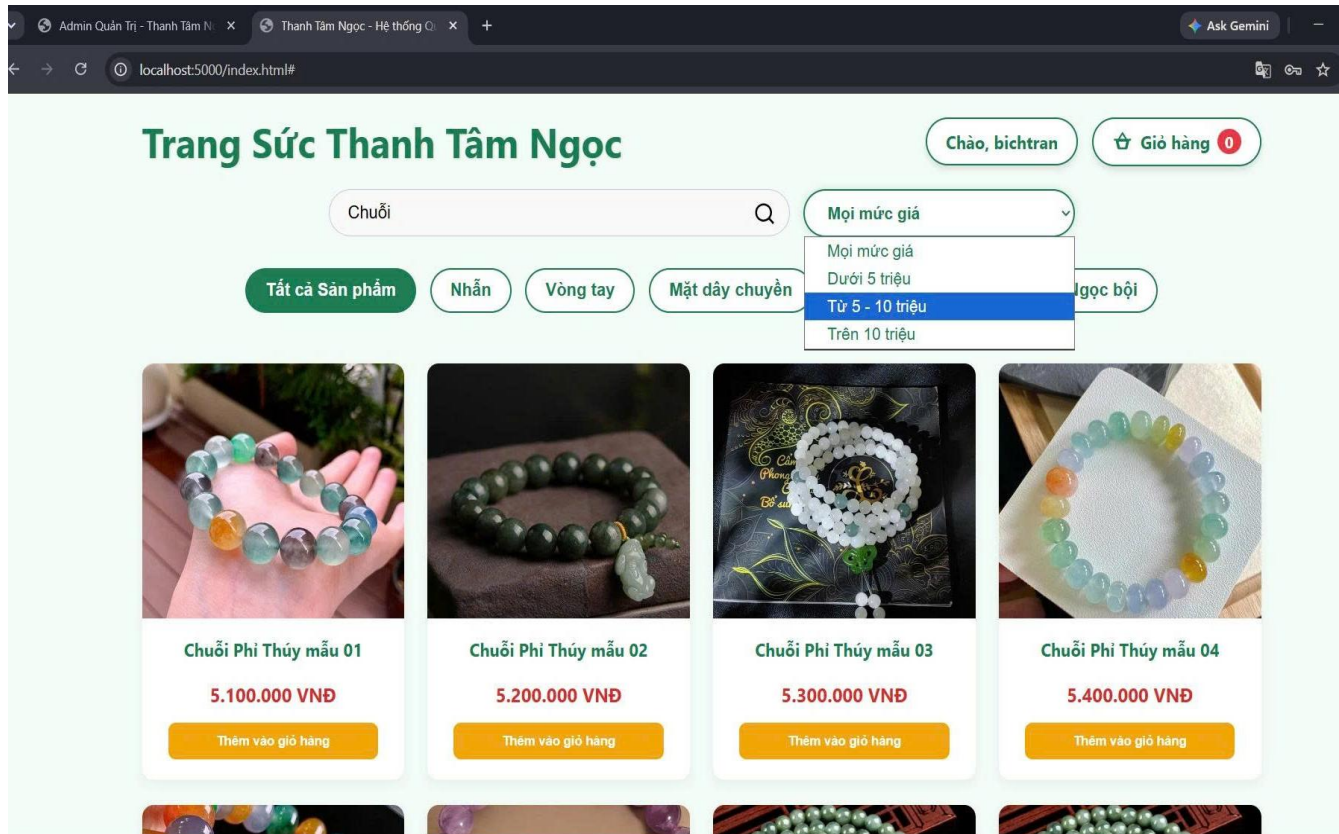
Trang chủ của website đóng vai trò là nơi trưng bày toàn bộ bộ sưu tập trang sức. Nhằm giúp khách hàng dễ dàng tiếp cận hơn 900 sản phẩm, hệ thống cung cấp thanh công cụ điều hướng đa chức năng:

- **Duyệt danh mục:** Khách hàng có thể chuyển đổi nhanh chóng giữa các bộ sưu tập (Nhẫn, Vòng tay, Mặt dây chuyền, Bông tai, Chuỗi, Ngọc bội). Giao diện lập tức cập nhật danh sách hiển thị tương ứng nhờ cơ chế bắt đồng bộ (Fetch API) mà không cần tải lại toàn bộ trang.

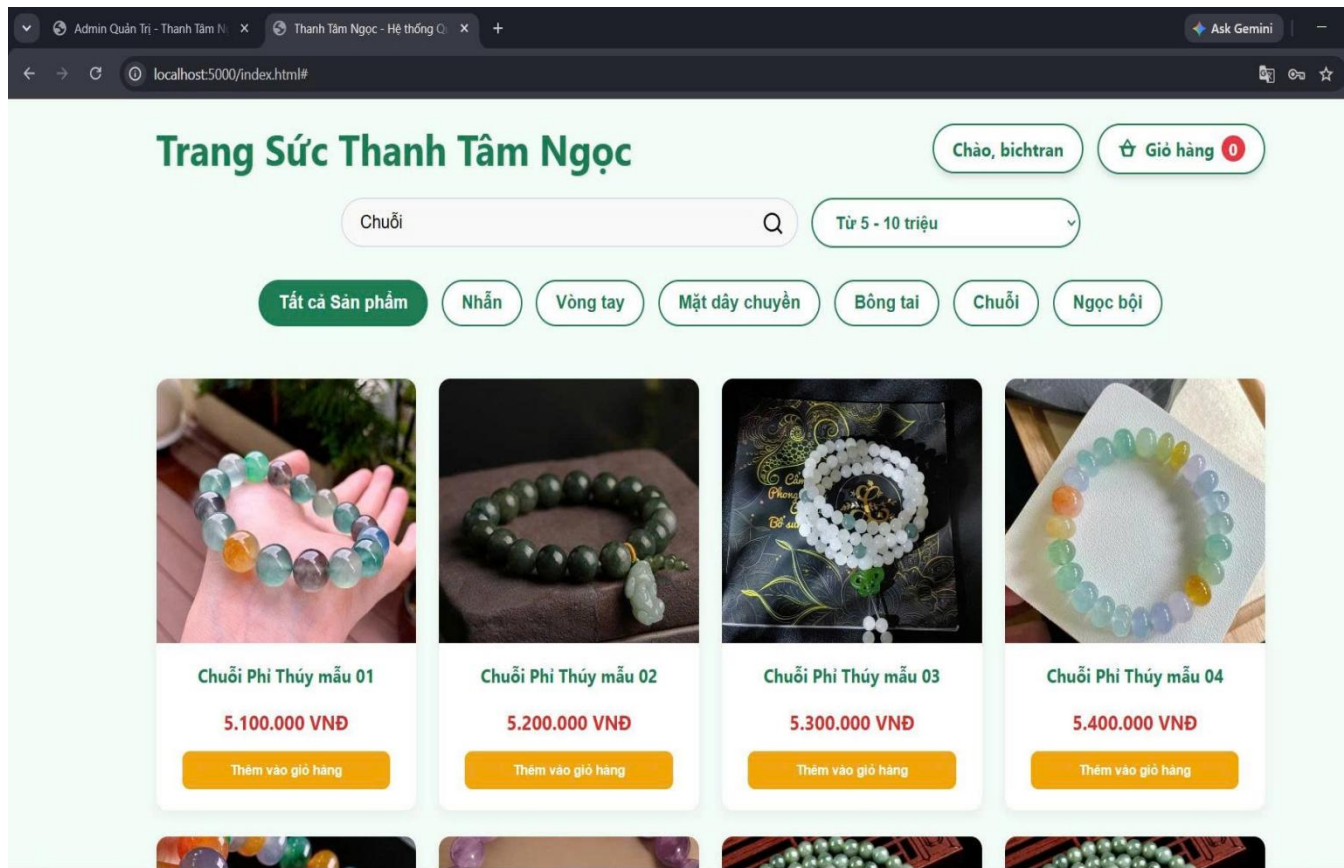


Hình 3.4. Duyệt các sản phẩm thuộc danh mục Mặt dây chuyền

- **Tìm kiếm và Lọc theo giá:** Để tối ưu hóa thời gian mua sắm, thanh tìm kiếm cho phép nhập từ khóa (ví dụ: "Chuỗi"). Kết hợp cùng với đó là bộ lọc phân khúc giá (Dưới 5 triệu, Từ 5 - 10 triệu, Trên 10 triệu), giúp khách hàng thu hẹp phạm vi kết quả theo đúng nhu cầu tài chính.

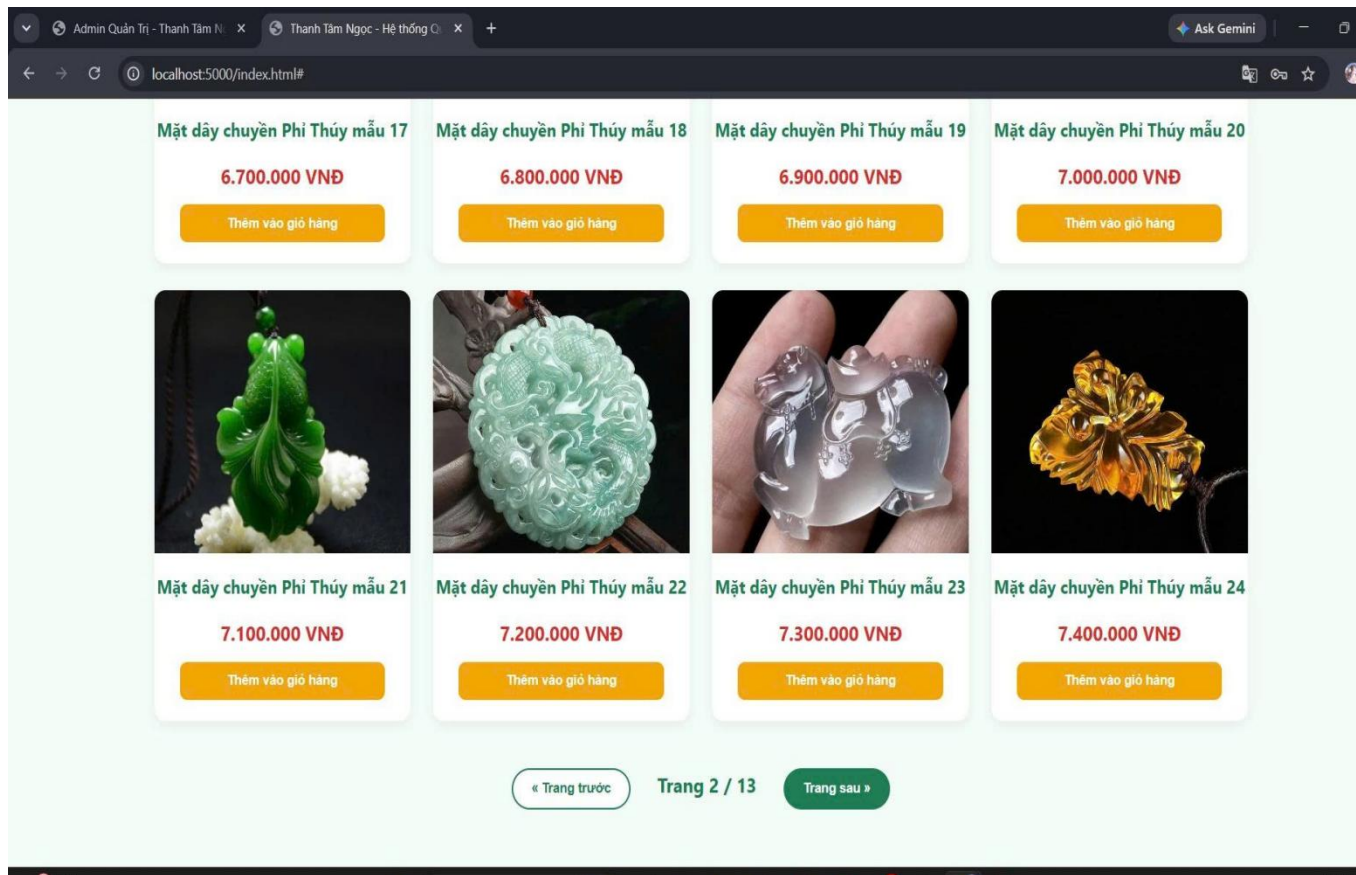


Hình 3.5. Kết quả tra cứu sản phẩm với từ khóa "Chuỗi"



Hình 3.6. Sử dụng Dropdown để lọc các sản phẩm theo phân khúc giá

- **Phân trang (Pagination):** Để tránh tình trạng quá tải dữ liệu (hơn 900 bản ghi) làm chậm tốc độ tải trang (page load speed) của trình duyệt, danh sách kết quả được chia nhỏ thành nhiều trang. Khách hàng sử dụng nút "Trang trước" và "Trang sau" để di chuyển mượt mà.



Hình 3.7. Nút điều hướng phân trang ở cuối danh sách sản phẩm

b. Xử lý nghiệp vụ (Back-end)

Nghiệp vụ trích xuất và lọc dữ liệu được xử lý tại tệp **ProductsController.cs**. Phương thức `GetProducts` sẽ tiếp nhận các tham số (query parameters) từ phía Client truyền lên bao gồm: `search` (từ khóa), `categoryId` (mã danh mục), `minPrice` và `maxPrice` (khoảng giá).

Tại đây, câu lệnh SQL truy vấn được xây dựng động (Dynamic SQL). Hệ thống bắt đầu với một câu lệnh cơ bản `SELECT * FROM Products WHERE 1=1` và lần lượt nối thêm các điều kiện (AND) nếu tham số đầu vào hợp lệ. Kỹ thuật sử dụng tham số hóa (`Parameters.AddWithValue`) được áp dụng triệt để nhằm ngăn chặn hoàn toàn rủi ro tấn công SQL Injection.

Đoạn mã nguồn xử lý truy vấn động (Trích từ tệp `ProductsController.cs`):

X

```
[HttpGet]
public IActionResult GetProducts([FromQuery] string search = "",
                                [FromQuery] int? categoryId = null, [FromQuery] decimal? minPrice = null,
                                [FromQuery] decimal? maxPrice = null)
{
    using (SqlConnection connection = new SqlConnection(_connectionString))
    {
        connection.Open();
        string sql = "SELECT * FROM Products WHERE 1=1";

        // Xây dựng câu lệnh truy vấn động dựa trên các tham số lọc
        if (!string.IsNullOrEmpty(search))
            sql += " AND Name LIKE @Search";
        if (categoryId.HasValue)
            sql += " AND CategoryId = @CategoryId";
        if (minPrice.HasValue)
            sql += " AND Price >= @MinPrice";
        if (maxPrice.HasValue)
            sql += " AND Price <= @MaxPrice";

        using (SqlCommand command = new SqlCommand(sql, connection))
        {
            // Truyền giá trị vào tham số để chống SQL Injection
            if (!string.IsNullOrEmpty(search))
                command.Parameters.AddWithValue("@Search", "%" + search + "%");
            if (categoryId.HasValue)
                command.Parameters.AddWithValue("@CategoryId", categoryId.Value);
            if (minPrice.HasValue)
                command.Parameters.AddWithValue("@MinPrice", minPrice.Value);
            if (maxPrice.HasValue)
                command.Parameters.AddWithValue("@MaxPrice", maxPrice.Value);

            using (SqlDataReader reader = command.ExecuteReader())
            {
                // Vòng lặp đọc dữ liệu và đổ vào danh sách trả về cho Front-end
                // ... (mã nguồn ánh xạ dữ liệu) ...
            }
        }
    }
}
```

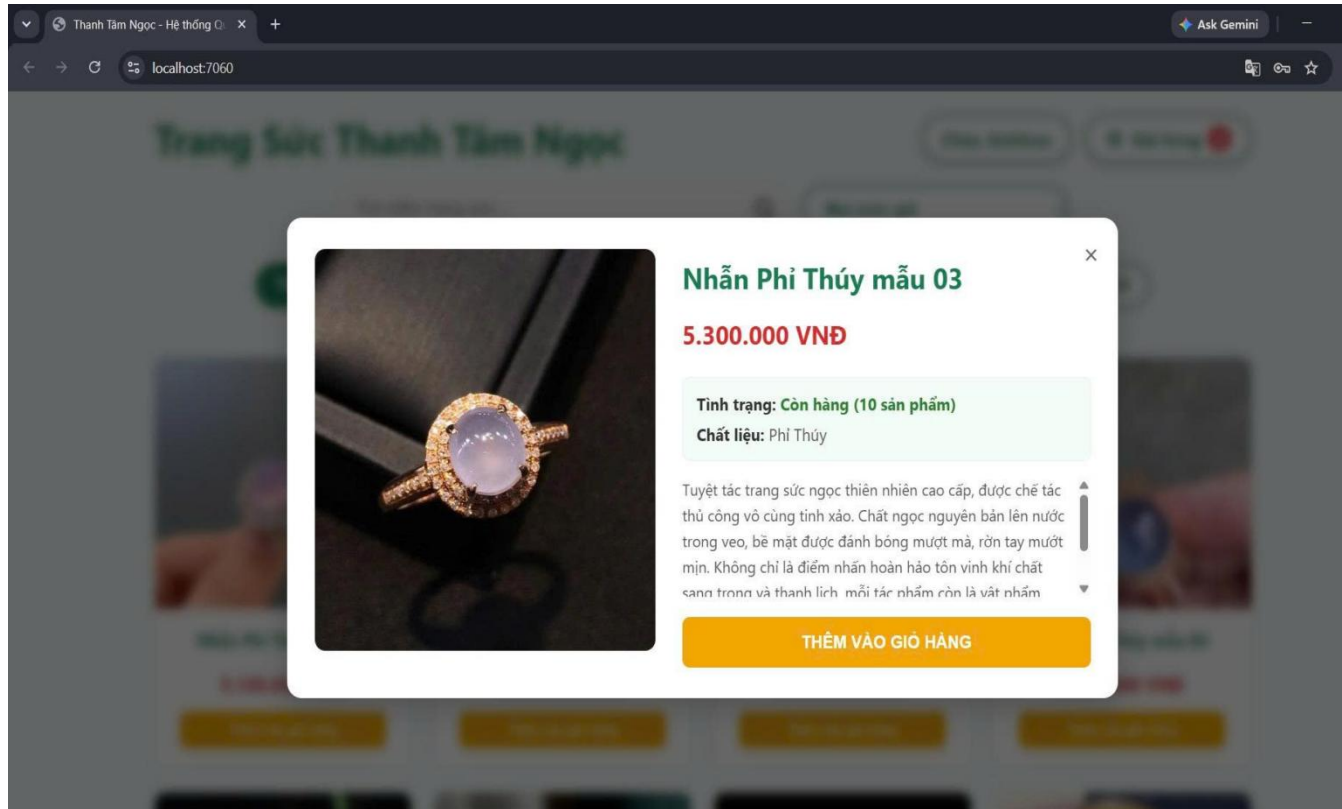
3.3. CHỨC NĂNG XEM CHI TIẾT SẢN PHẨM

a. Giao diện người dùng (Front-end)

Nhằm cung cấp thông tin minh bạch và đầy đủ nhất trước khi khách hàng quyết định đưa vào giỏ hàng, hệ thống được thiết kế chức năng xem chi tiết sản phẩm. Khi người dùng nhấp chuột (click) vào bất kỳ hình ảnh sản phẩm nào trên danh sách, một hộp thoại nổi (Modal) sẽ lập tức xuất hiện.

Hộp thoại này được trình bày trực quan, kết hợp hình ảnh cận cảnh sắc nét ở bên trái và thông tin chi tiết ở bên phải. Khách hàng có thể kiểm tra kỹ lưỡng

các thuộc tính quan trọng như: Giá bán hiện tại, Tình trạng tồn kho, Chất liệu ngọc, và đặc biệt là đoạn văn bản miêu tả nét độc đáo, ý nghĩa phong thủy của từng tác phẩm.



Hình 3.8. Giao diện hộp thoại xem thông tin chi tiết của Nhẫn Phi Thúy

b. Xử lý nghiệp vụ (Back-end)

Khi có sự kiện click vào sản phẩm, Front-end sẽ gửi một mã định danh (Id) của sản phẩm đó về cho máy chủ. Nghiệp vụ này được đảm nhận bởi phương thức `GetProductById` trong tệp `ProductsController.cs`.

Hệ thống sẽ thực hiện truy vấn trực tiếp vào bảng `Products` trong cơ sở dữ liệu dựa trên khóa chính (Id). Câu lệnh SQL đơn giản nhưng hiệu quả: `SELECT * FROM Products WHERE Id = @Id`. Nhờ sử dụng kết hợp `SqlConnection` và `SqlCommand` có truyền tham số bảo mật, hệ thống trích xuất toàn bộ trường dữ liệu của đúng sản phẩm đó và đóng gói thành chuỗi định dạng JSON trả về cho giao diện hiển thị.

Đoạn mã nguồn trích xuất chi tiết sản phẩm (Trích từ tệp ProductsController.cs):

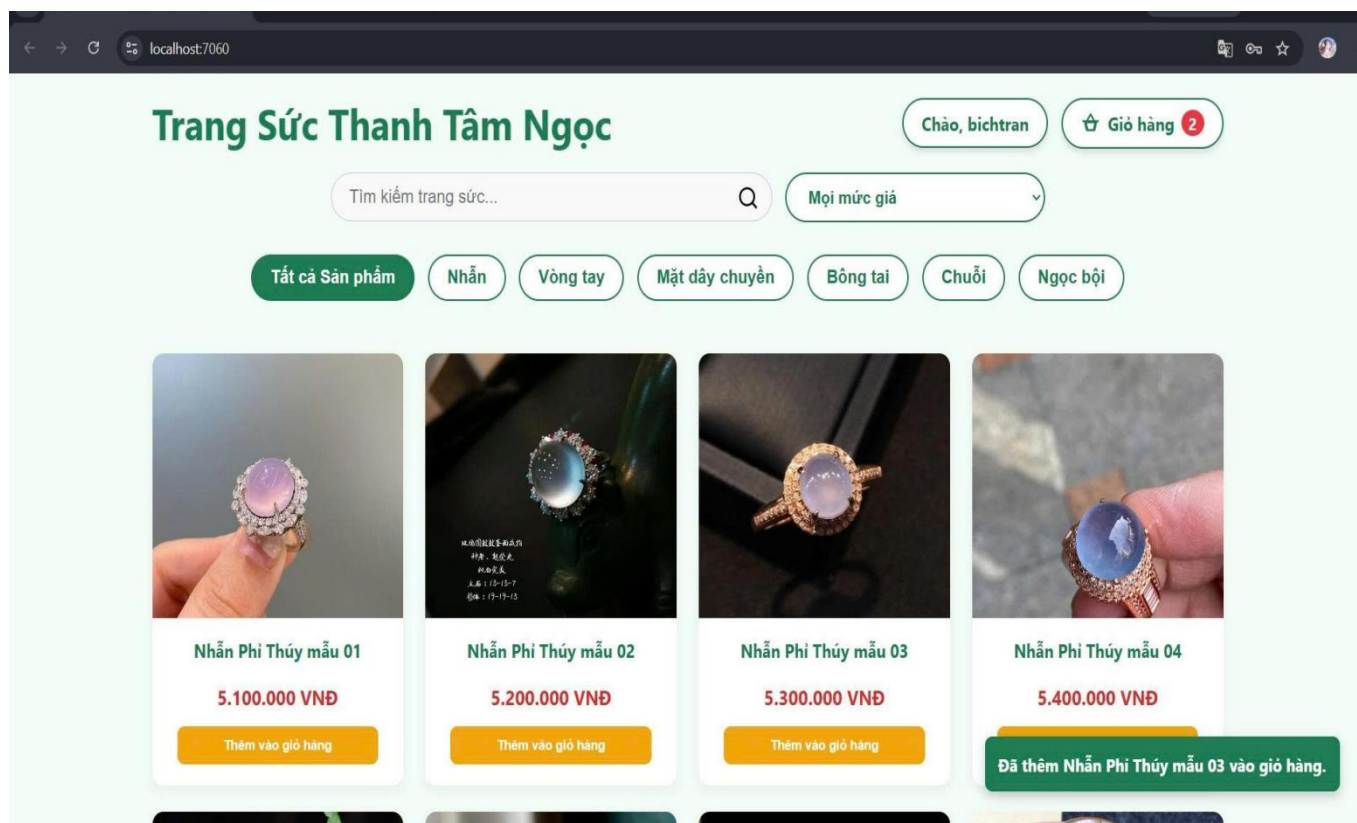
```
[HttpGet("{id}")]
public IActionResult GetProductById(int id)
{
    try
    {
        using (SqlConnection connection = new SqlConnection(_connectionString))
        {
            connection.Open();
            string sql = "SELECT * FROM Products WHERE Id = @Id";
            using (SqlCommand command = new SqlCommand(sql, connection))
            {
                // Truyền mã sản phẩm an toàn vào câu truy vấn
                command.Parameters.AddWithValue("@Id", id);

                using (SqlDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        var product = new
                        {
                            id = reader["Id"],
                            name = reader["Name"],
                            price = reader["Price"],
                            imageUrl = reader["ImageUrl"] != DBNull.Value ? reader["ImageUrl"].ToString() : "",
                            gemType = reader["GemType"] != DBNull.Value ? reader["GemType"].ToString() : "",
                            description = reader["Description"] != DBNull.Value ? reader["Description"].ToString() : "",
                            stockQuantity = reader["StockQuantity"]
                        };
                        // Trả về dữ liệu chi tiết dạng JSON
                        return Ok(product);
                    }
                    else
                    {
                        return NotFound("Không tìm thấy thông tin sản phẩm.");
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        return StatusCode(500, "Lỗi Server: " + ex.Message);
    }
}
```

3.4. CHỨC NĂNG GIỎ HÀNG (SHOPPING CART)

a. Giao diện người dùng (Front-end)

Giỏ hàng là chức năng đóng vai trò "chốt chặn" quan trọng nhất trước khi khách hàng quyết định thanh toán. Để tăng tính tương tác, khi người dùng bấm vào nút "Thêm vào giỏ hàng" tại trang chủ hoặc trang chi tiết, hệ thống lập tức hiển thị thông báo "Đã thêm... vào giỏ hàng" bằng một hộp thoại nhỏ (toast notification) góc dưới màn hình. Đồng thời, biểu tượng giỏ hàng trên thanh điều hướng cũng tự động cập nhật số lượng (Animation hiệu ứng bay vào giỏ).



Hình 3.9. Thông báo trực quan khi khách hàng thêm sản phẩm vào giỏ

Bên trong giao diện Giỏ hàng (tích hợp cùng trang Thanh toán), người dùng có thể quản lý các món trang sức đã chọn. Bảng tóm tắt đơn hàng cho phép thực hiện các thao tác: tăng/giảm số lượng sản phẩm (+, -) hoặc xóa sản phẩm khỏi

giỏ (x). Mọi thay đổi về số lượng sẽ ngay lập tức kích hoạt tính toán lại trường **Tạm tính** và **Tổng cộng** một cách chính xác mà không cần tải lại trang.

Hình 3.10. Giao diện quản lý chi tiết các sản phẩm trong Giỏ hàng

b. Xử lý nghiệp vụ (Back-end)

Nghiệp vụ quản lý Giỏ hàng được thiết kế chặt chẽ thông qua tệp **CartController.cs**. Hệ thống sử dụng Entity Framework Core để thao tác với cơ sở dữ liệu và yêu cầu người dùng phải đăng nhập (thông qua cơ chế bảo mật JWT Token - [Authorize]).

Khi có yêu cầu thêm sản phẩm, phương thức AddToCart sẽ trích xuất mã định danh khách hàng (userId). Hệ thống kiểm tra xem người dùng đã có Giỏ hàng (Cart) nào tồn tại chưa, nếu chưa sẽ tự động tạo mới. Tiếp theo, mã nguồn sẽ dò tìm sản phẩm trong chi tiết giỏ hàng (CartItem):

- Nếu sản phẩm **đã tồn tại**, hệ thống chỉ thực hiện lệnh cộng dồn số lượng (Quantity).
- Nếu sản phẩm **chưa tồn tại**, hệ thống sẽ khởi tạo một bản ghi CartItem mới. Cuối cùng, phương thức SaveChangesAsync() được gọi để lưu toàn bộ thay đổi xuống CSDL SQL Server một cách toàn vẹn.

Đoạn mã nguồn xử lý thêm sản phẩm vào giỏ (Trích từ tệp

CartController.cs):

```
[HttpPost("add")]
public async Task<ActionResult> AddToCart([FromBody] CartItemRequest request)
{
    // Xác thực người dùng qua Token
    var userId = GetCurrentUser();

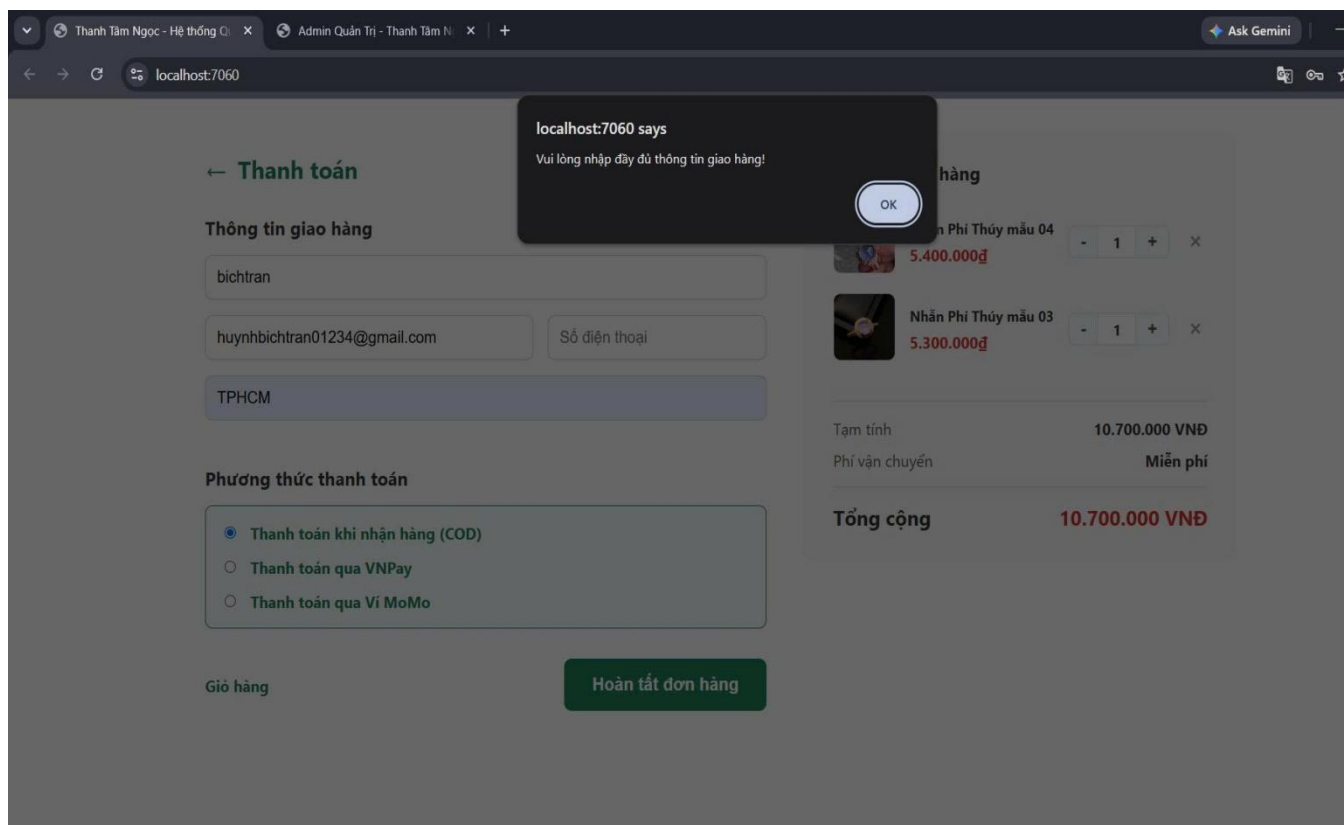
    // Lấy giỏ hàng của khách hàng hiện tại
    var cart = await _context.Carts.Include(c => c.CartItems).FirstOrDefaultAsync(c => c.UserId == userId);
    if (cart == null)
    {
        cart = new Cart { UserId = userId };
        _context.Carts.Add(cart);
        await _context.SaveChangesAsync();
    }

    // Kiểm tra sản phẩm đã có trong giỏ chưa
    var existingItem = cart.CartItems?.FirstOrDefault(ci => ci.ProductId == request.ProductId);
    if (existingItem != null)
    {
        // Nếu đã có, tiến hành cộng dồn số lượng
        existingItem.Quantity += request.Quantity;
    }
    else
    {
        // Nếu chưa có, thêm sản phẩm mới vào chi tiết giỏ hàng
        var newItem = new CartItem
        {
            CartId = cart.Id,
            ProductId = request.ProductId,
            Quantity = request.Quantity
        };
        _context.CartItems.Add(newItem);
    }

    // Lưu thay đổi vào CSDL
    await _context.SaveChangesAsync();
    return Ok(new { message = "Đã bỏ ngọc quý vào giỏ thành công rực rỡ!" });
}
```

3.5. CHỨC NĂNG THANH TOÁN VÀ ĐẶT HÀNG (CHECKOUT)

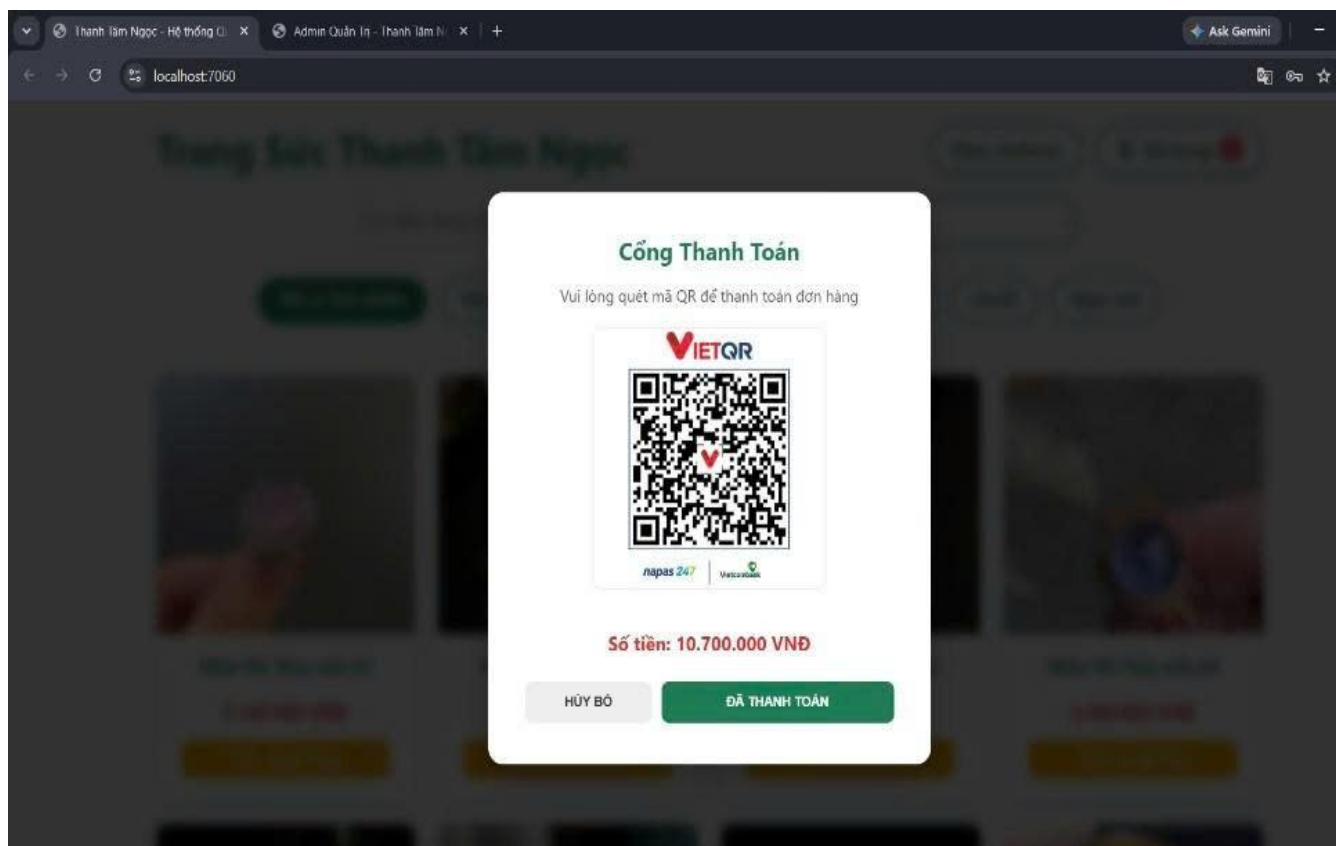
a. Giao diện người dùng (Front-end) Trang thanh toán được thiết kế liền mạch với giỏ hàng, yêu cầu người dùng điền đầy đủ thông tin giao hàng (Tên, Email, Số điện thoại, Địa chỉ). Hệ thống có tích hợp bộ kiểm tra tính hợp lệ (Validation). Nếu khách hàng bỏ trống bất kỳ trường thông tin bắt buộc nào, một hộp thoại cảnh báo sẽ lập tức xuất hiện để ngăn chặn việc gửi dữ liệu lỗi về máy chủ.



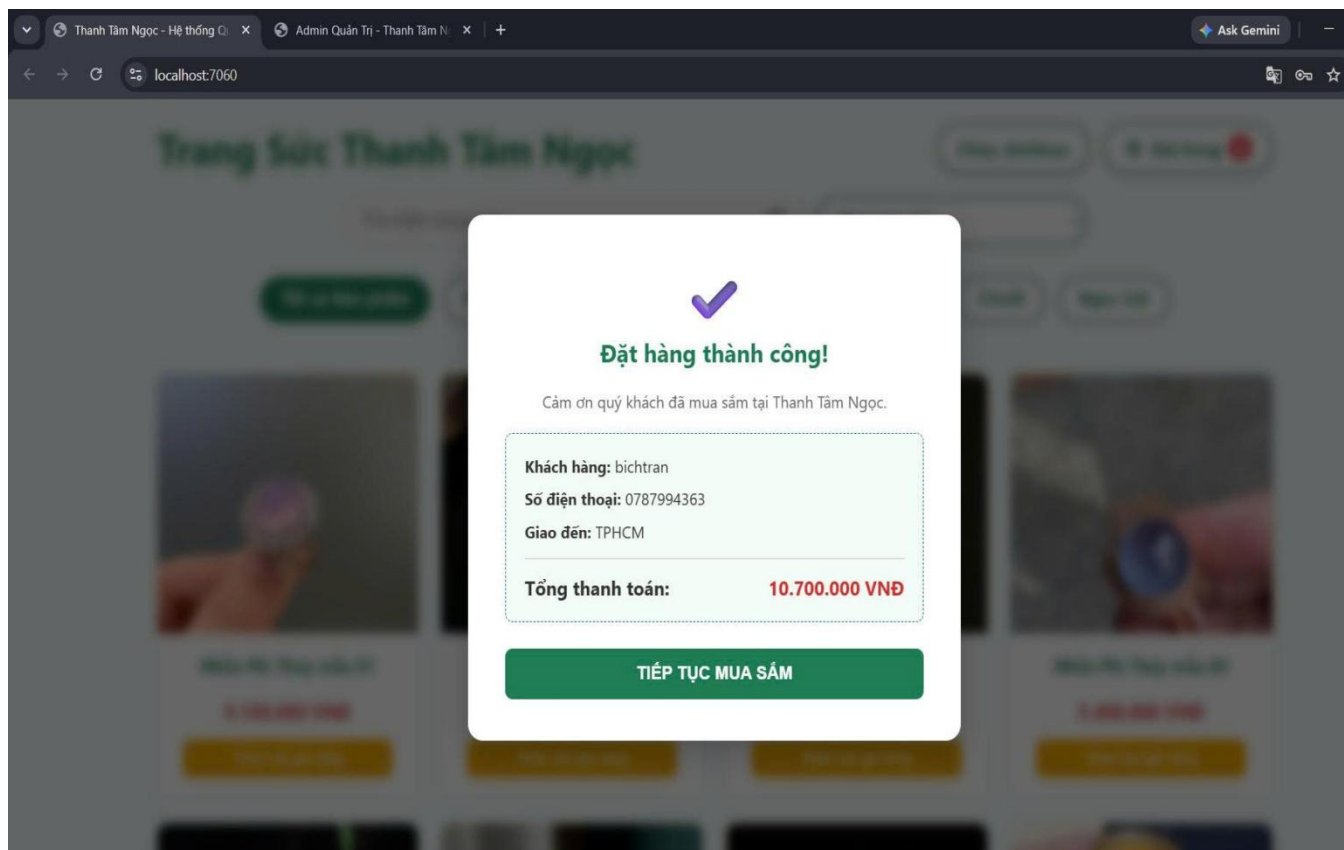
Hình 3.11. Cảnh báo khi người dùng nhập thiếu thông tin giao hàng

Website hỗ trợ đa dạng phương thức thanh toán nhằm tối đa hóa sự tiện lợi cho khách hàng, bao gồm hai luồng xử lý chính:

1. **Thanh toán khi nhận hàng (COD):** Khách hàng chỉ cần bấm "Hoàn tất đơn hàng", hệ thống sẽ xử lý và hiển thị ngay thông báo đặt hàng thành công.
2. **Thanh toán trực tuyến (VNPay / Ví MoMo):** Khi chọn phương thức này, một cổng thanh toán giả lập sẽ hiện lên dưới dạng hộp thoại chứa mã VietQR tương ứng với tổng số tiền của đơn hàng. Sau khi khách hàng thực hiện thao tác quét mã và bấm xác nhận "Đã thanh toán", hệ thống mới tiến hành ghi nhận đơn.



Hình 3.12. *Giao diện cổng thanh toán tích hợp mã VietQR trực tuyến*



Hình 3.13. Thông báo xác nhận đặt hàng thành công và kết thúc quy trình

b. Xử lý nghiệp vụ (Back-end) Quy trình ghi nhận đơn hàng vào cơ sở dữ liệu được thực hiện tại tệp **OrdersController.cs** thông qua API [HttpPost("checkout")].

Đây là một nghiệp vụ phức tạp đòi hỏi tính toàn vẹn dữ liệu cao, do đó hệ thống sẽ thực hiện lưu trữ theo hai bước (Master-Detail). Đầu tiên, mã nguồn sử dụng lệnh INSERT kết hợp với mệnh đề OUTPUT INSERTED.OrderId để lưu thông tin chung của hóa đơn vào bảng Orders và lập tức lấy lại mã hóa đơn vừa được tạo (OrderId). Sau đó, hệ thống sử dụng một vòng lặp foreach duyệt qua danh sách các sản phẩm có trong giỏ hàng để tiếp tục lưu chi tiết số lượng, đơn giá của từng món vào bảng phụ OrderDetails.

Đoạn mã nguồn xử lý thanh toán và tạo hóa đơn (Trích từ tệp OrdersController.cs):

```
[HttpPost("checkout")]
public IActionResult Checkout([FromBody] CheckoutRequest request)
{
    try
    {
        using (SqlConnection conn = new SqlConnection(_connectionString))
        {
            conn.Open();

            // Bước 1: Lưu thông tin chung của đơn hàng vào bảng Orders
            string orderQuery = @"
                INSERT INTO Orders (UserEmail, ShippingAddress, TotalAmount, OrderStatus)
                OUTPUT INSERTED.OrderId
                VALUES (@Email, @Address, @TotalAmount, N'Thành công')";

            int orderId = 0;
            using (SqlCommand cmd = new SqlCommand(orderQuery, conn))
            {
                cmd.Parameters.AddWithValue("@Email", request.Email);
                cmd.Parameters.AddWithValue("@Address", request.Address);
                cmd.Parameters.AddWithValue("@TotalAmount", request.TotalAmount);
                // Lấy mã OrderId vừa được hệ thống tự động sinh ra
                orderId = (int)cmd.ExecuteScalar();
            }

            // Bước 2: Lưu chi tiết từng sản phẩm vào bảng OrderDetails
            foreach (var item in request.Items)
            {
                string detailQuery = @"
                    INSERT INTO OrderDetails (OrderId, ProductName, Quantity, UnitPrice)
                    VALUES (@OrderId, @ProductName, @Quantity, @UnitPrice)";

                using (SqlCommand detailCmd = new SqlCommand(detailQuery, conn))
                {
                    detailCmd.Parameters.AddWithValue("@OrderId", orderId);
                    detailCmd.Parameters.AddWithValue("@ProductName", item.Name);
                    detailCmd.Parameters.AddWithValue("@Quantity", item.Quantity);
                    detailCmd.Parameters.AddWithValue("@UnitPrice", item.Price);
                    detailCmd.ExecuteNonQuery();
                }
            }
        }
        return Ok(new { message = "Thành công" });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { message = "Lỗi hệ thống: " + ex.Message });
    }
}
```

3.6. QUẢN LÝ HỒ SƠ CÁ NHÂN VÀ LỊCH SỬ MUA HÀNG

a. Giao diện người dùng (Front-end)

Sau khi đăng nhập thành công, khách hàng có thể truy cập vào khu vực quản trị tài khoản cá nhân. Giao diện được chia thành hai phân hệ chính để dễ dàng thao tác:

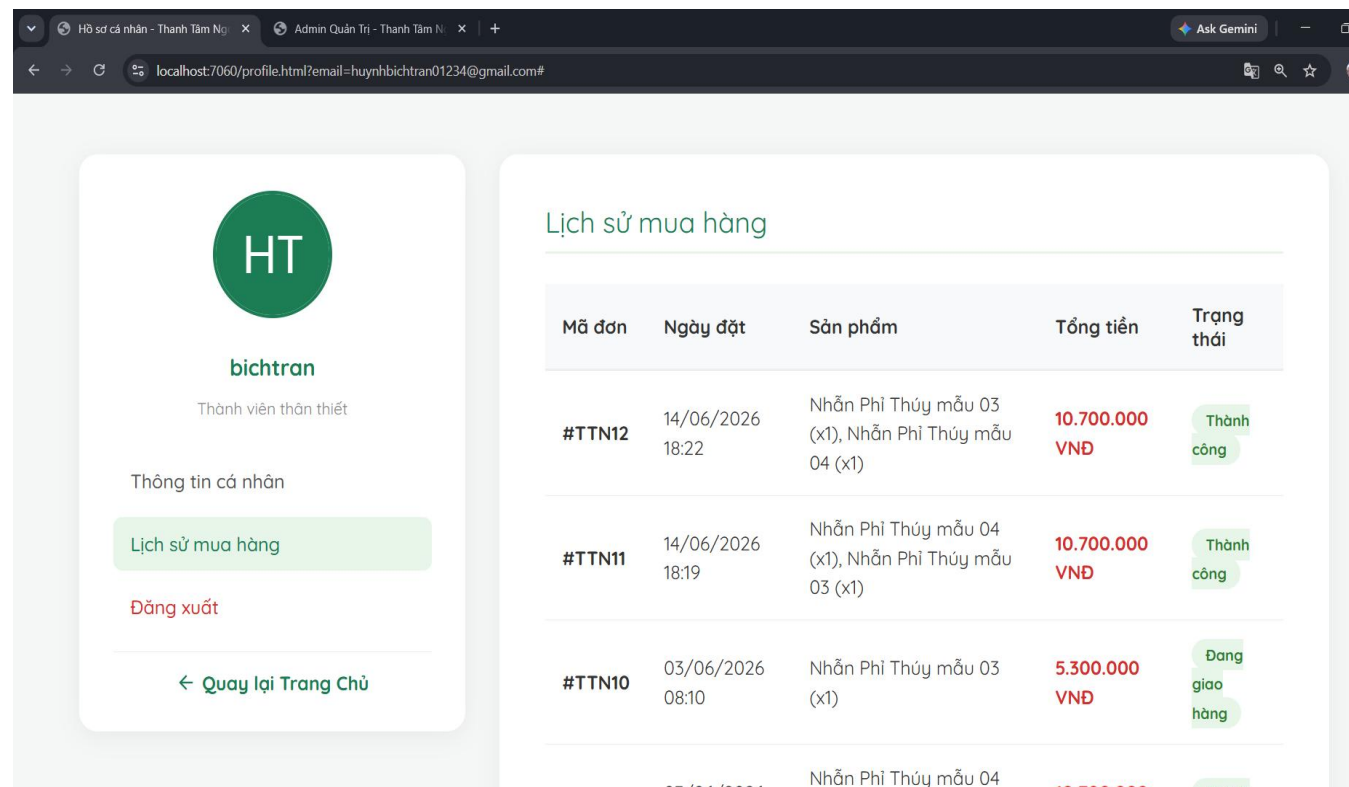
1. **Thông tin cá nhân:** Cho phép khách hàng xem và cập nhật các thông tin liên hệ như Số điện thoại, Ngày sinh và Địa chỉ nhận hàng mặc định. Dữ liệu này sẽ được tự động điền vào form thanh toán để tiết kiệm thời gian cho người dùng ở những lần mua sắm sau.

The screenshot displays a web application interface for user profile management. The browser address bar shows the URL: localhost:7060/profile.html?email=huynhbichtran01234@gmail.com#. The interface is split into two columns. The left column features a user profile card with a green circular avatar labeled 'HT', the name 'bichtran', and the title 'Thành viên thân thiết'. Below the card are three links: 'Thông tin cá nhân' (highlighted in green), 'Lịch sử mua hàng', and 'Đăng xuất'. A green button 'Quay lại Trang Chủ' is at the bottom. The right column is titled 'Hồ sơ cá nhân' and contains several form fields: 'Họ và Tên' (bichtran), 'Số điện thoại' (0787994363), 'Email' (huynhbichtran01234@gmail.com), 'Ngày sinh' (mm/dd/yyyy), and 'Địa chỉ nhận hàng' (117/28 xvnt p17, quận Bình Thạnh, Tp HCM). A green button 'Cập nhật thông tin' is located at the bottom right of the right column.

Hình 3.14. Giao diện quản lý thông tin và cập nhật hồ sơ cá nhân

2. **Lịch sử mua hàng:** Cung cấp bảng thống kê chi tiết toàn bộ các đơn hàng đã đặt trong quá khứ. Khách hàng dễ dàng theo dõi Mã đơn, Ngày đặt,

danh sách các Sản phẩm đi kèm số lượng, Tổng tiền và Trạng thái hiện tại của đơn hàng (Đang chờ xử lý, Đang giao hàng, Thành công).



Hình 3.15. Giao diện theo dõi danh sách lịch sử đơn hàng của khách hàng

b. Xử lý nghiệp vụ (Back-end)

Dữ liệu hiển thị trên trang cá nhân được trích xuất từ hai nguồn khác nhau. Thông tin người dùng do AccountController.cs đảm nhận, trong khi lịch sử đơn hàng được truy xuất thông qua API [HttpGet("history/{email}")] tại tệp **OrdersController.cs**.

Điểm nhấn kỹ thuật ở đây là câu lệnh truy vấn SQL tích hợp hàm STRING_AGG (String Aggregation). Thay vì phải gọi nhiều lệnh truy vấn phức tạp hoặc xử lý vòng lặp nặng nề ở phía C#, hệ thống đã khéo léo gộp toàn bộ tên sản phẩm và số lượng tương ứng từ bảng OrderDetails thành một chuỗi

duy nhất ngay từ dưới CSDL, giúp tối ưu hóa hiệu suất (performance) và giảm thiểu băng thông truyền tải.

Đoạn mã nguồn truy xuất lịch sử đơn hàng (Trích từ tệp OrdersController.cs):

```
[HttpGet("history/{email}")]
public IActionResult GetOrderHistory(string email)
{
    try
    {
        var orders = new List<object>();
        using (SqlConnection conn = new SqlConnection(_connectionString))
        {
            conn.Open();
            // Sử dụng câu lệnh lồng và STRING_AGG để gộp chi tiết sản phẩm
            string query = @"
                SELECT o.OrderId, o.OrderDate, o.TotalAmount, o.OrderStatus,
                       (SELECT STRING_AGG(ProductName + ' (x' + CAST(Quantity AS VARCHAR(10)) + ')', ', ')
                        FROM OrderDetails od WHERE od.OrderId = o.OrderId) AS Products
                FROM Orders o
                WHERE o.UserEmail = @Email
                ORDER BY o.OrderDate DESC";

            using (SqlCommand cmd = new SqlCommand(query, conn))
            {
                // Truyền tham số Email để xác định đúng khách hàng
                cmd.Parameters.AddWithValue("@Email", email);
                using (SqlDataReader reader = cmd.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        orders.Add(new
                        {
                            OrderId = reader["OrderId"].ToString(),
                            OrderDate = Convert.ToDateTime(reader["OrderDate"]).ToString("dd/MM/yyyy HH:mm"),
                            Products = reader["Products"].ToString(),
                            TotalAmount = reader["TotalAmount"],
                            Status = reader["OrderStatus"].ToString()
                        });
                    }
                }
            }
        }
        return Ok(orders);
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { message = "Lỗi hệ thống: " + ex.Message });
    }
}
```

3.7. TỐI ƯU TRẢI NGHIỆM NGƯỜI DÙNG (UX/UI)

Bên cạnh việc hoàn thiện các tính năng cốt lõi, hệ thống Thanh Tâm Ngọc còn đặc biệt chú trọng đến việc tối ưu hóa Trải nghiệm người dùng (UX) trong mọi kịch bản thao tác, tuân thủ các nguyên tắc thiết kế web hiện đại:

- **Trạng thái xử lý (Loading/Empty/Error States):** * **Loading State:** Khi dữ liệu đang được truy xuất từ cơ sở dữ liệu qua Fetch API, hệ thống luôn hiển thị trạng thái "Đang tải dữ liệu..." để khách hàng không bị hoang mang khi chờ đợi.
- **Empty State:** Trong trường hợp lịch sử mua hàng trống hoặc tìm kiếm không có kết quả, giao diện được xử lý gọn gàng bằng các thông báo điều hướng thân thiện thay vì để khoảng trống.
- **Error State:** Hệ thống kiểm soát lỗi (validation) cực kỳ chặt chẽ ở các biểu mẫu (Form). Nếu người dùng nhập sai mật khẩu hoặc thiếu thông tin thanh toán, hệ thống sẽ lập tức chặn lại và hiển thị cảnh báo hộp thoại trực quan (như đã trình bày ở Hình 3.11).
- **Hiệu ứng chuyển động (Animation):** Phản hồi thị giác là yếu tố cực kỳ quan trọng. Khi người dùng click "Thêm vào giỏ hàng", một hộp thoại Toast Notification được thiết kế với hiệu ứng trượt mượt mà sẽ xuất hiện (như Hình 3.9), giúp người dùng nhận biết thao tác đã thành công mà không gây gián đoạn luồng mua sắm.
- **Tính năng Làm mới (Pull-to-refresh / Reload):** Đối với phân hệ Quản trị viên (Admin), thay vì phải F5 tải lại toàn bộ trang, quản trị viên được cung cấp nút thao tác "Làm mới" (Reload data) gọi trực tiếp API để cập nhật tiến độ đơn hàng mới nhất theo thời gian thực một cách nhanh chóng.

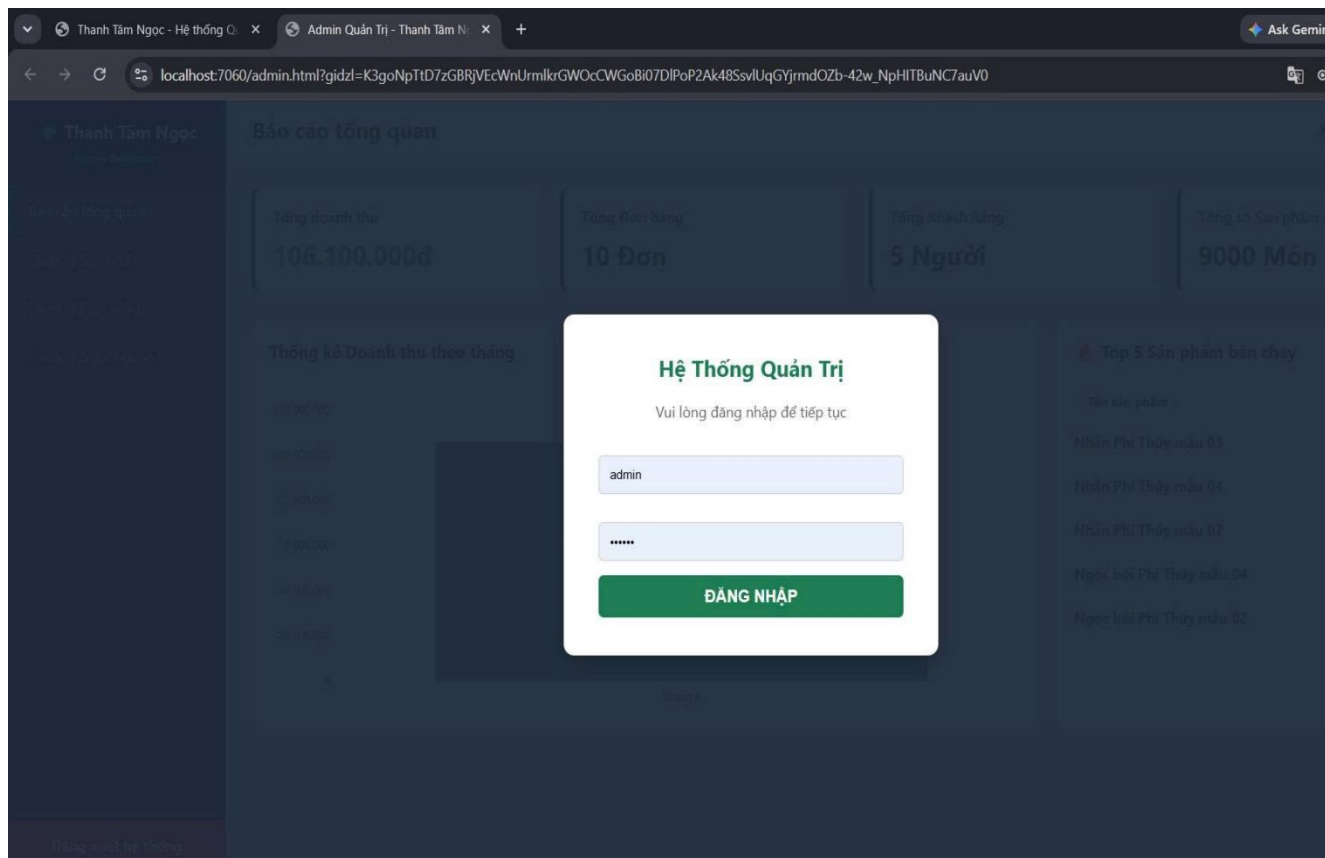
3.8. PHÂN HỆ QUẢN TRỊ VIÊN (ADMIN DASHBOARD)

3.8.1. Chức năng Đăng nhập quản trị

a. Giao diện người dùng (Front-end)

Phân hệ Quản trị (Admin Dashboard) được thiết kế tại một trang web hoàn toàn riêng biệt (admin.html) nhằm đảm bảo tính bảo mật và tách biệt luồng nghiệp vụ với khách hàng.

Để ngăn chặn các truy cập trái phép, khi vừa tải trang, một lớp mặt nạ (overlay modal) chứa biểu mẫu đăng nhập sẽ bao phủ toàn bộ màn hình. Người dùng bắt buộc phải điền chính xác Tên đăng nhập và Mật khẩu được hệ thống cấp phép thì mới có thể tiến vào bảng điều khiển bên trong.



Hình 3.16. Giao diện yêu cầu Đăng nhập bắt buộc tại trang Quản trị

b. Xử lý nghiệp vụ (Logic)

Nghiệp vụ kiểm tra xác thực được quản lý tại tệp giao diện **admin.html** thông qua ngôn ngữ JavaScript. Khi người dùng nhấn "ĐĂNG NHẬP", hàm loginAdmin() sẽ thu thập dữ liệu từ các trường input và tiến hành đối chiếu.

- Nếu thông tin không chính xác: Hệ thống lập tức hiển thị cảnh báo (alert) từ chối quyền truy cập.
- Nếu thông tin hợp lệ: Giao diện đăng nhập (#admin-login) sẽ được ẩn đi. Ngay khoảnh khắc đó, hệ thống sẽ tự động kích hoạt một loạt các hàm Fetch API để gọi xuống Back-end, tiến hành nạp toàn bộ dữ liệu (Thống kê, Sản phẩm, Đơn hàng, Người dùng) đổ ra màn hình cho Quản trị viên.

Đoạn mã nguồn xử lý kiểm tra Đăng nhập Admin (Trích từ tệp admin.html):

```
function loginAdmin() {  
    const user = document.getElementById('admin-user').value;  
    const pass = document.getElementById('admin-pass').value;  
  
    // Đối chiếu thông tin tài khoản quản trị  
    if (user === 'admin' && pass === '123456') {  
        // Ẩn hộp thoại đăng nhập khi xác thực thành công  
        document.getElementById('admin-login').style.display = 'none';  
  
        // Tự động gọi Fetch API để nạp dữ liệu cho toàn bộ Dashboard  
        loadDashboardStats();  
        loadAdminProducts();  
        loadAdminOrders();  
        loadAdminUsers();  
        loadChartAndTop();  
    } else {  
        alert('Sai tên đăng nhập hoặc mật khẩu quản trị!');  
    }  
}
```

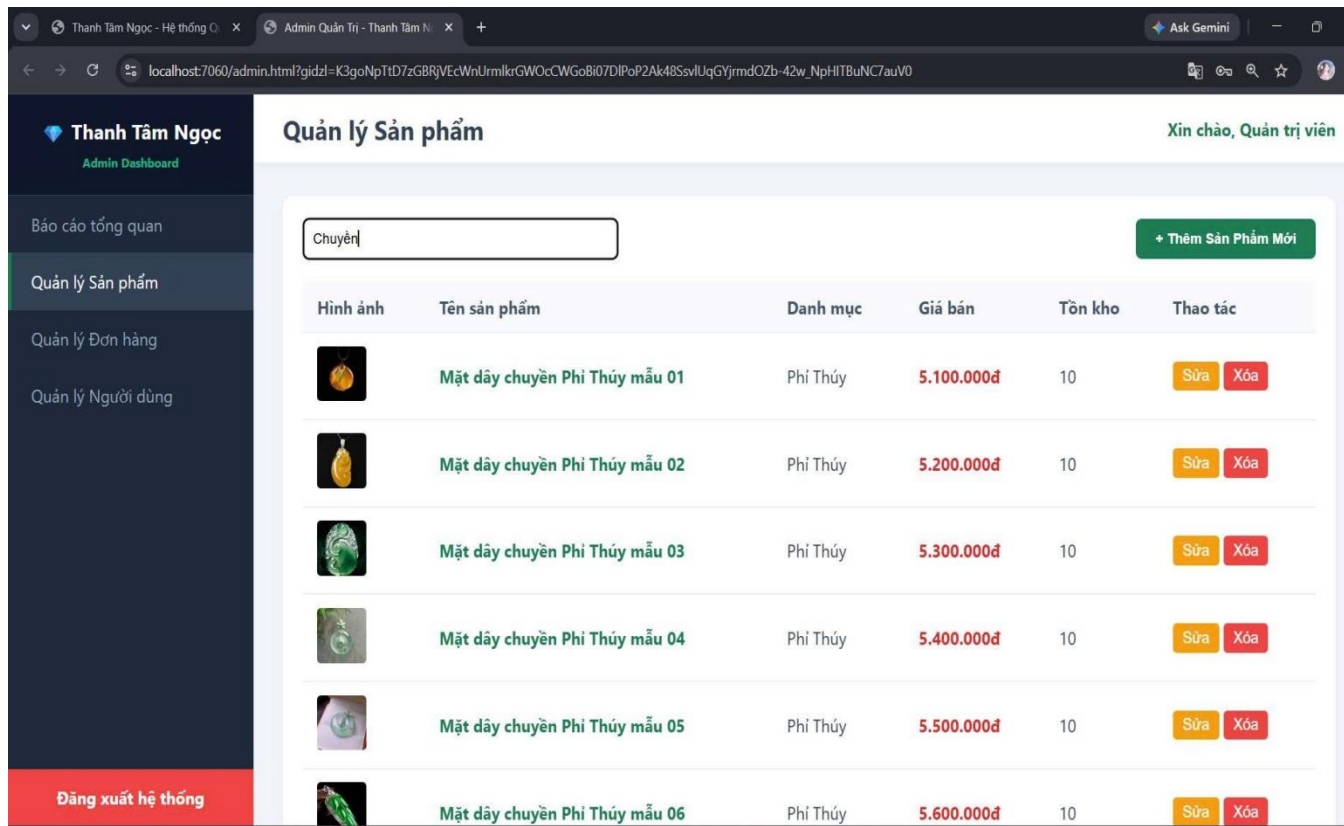
3.8.2. Chức năng Quản lý Sản phẩm

a. Giao diện người dùng (Front-end) Đây là phân hệ cốt lõi giúp Quản trị viên kiểm soát toàn bộ kho hàng của hệ thống. Giao diện được thiết kế dưới dạng bảng dữ liệu (Data Table), cung cấp cái nhìn tổng quan về danh sách sản phẩm bao gồm: Hình ảnh thu nhỏ, Tên sản phẩm, Danh mục, Giá bán, và Số lượng Tồn kho.

Hình ảnh	Tên sản phẩm	Danh mục	Giá bán	Tồn kho	Thao tác
	Nhẫn Phi Thúy mẫu 01	Phi Thúy	5.100.000đ	10	<button>Sửa</button> <button>Xóa</button>
	Nhẫn Phi Thúy mẫu 02	Phi Thúy	5.200.000đ	10	<button>Sửa</button> <button>Xóa</button>
	Nhẫn Phi Thúy mẫu 03	Phi Thúy	5.300.000đ	10	<button>Sửa</button> <button>Xóa</button>
	Nhẫn Phi Thúy mẫu 04	Phi Thúy	5.400.000đ	10	<button>Sửa</button> <button>Xóa</button>
	Nhẫn Phi Thúy mẫu 05	Phi Thúy	5.500.000đ	10	<button>Sửa</button> <button>Xóa</button>
	Nhẫn Phi Thúy mẫu 06	Phi Thúy	5.600.000đ	10	<button>Sửa</button> <button>Xóa</button>

Hình 3.17. Giao diện tổng quan danh sách Quản lý kho Sản phẩm

Để hỗ trợ việc tra cứu trong một cơ sở dữ liệu lớn (hơn 900 sản phẩm), hệ thống cung cấp một thanh tìm kiếm thời gian thực. Ngay khi quản trị viên nhập từ khóa (ví dụ: "Chuyên"), danh sách sẽ lập tức được lọc và chỉ hiển thị các mặt hàng có chứa cụm từ tương ứng.



Hình 3.18. Quản trị viên sử dụng thanh tìm kiếm để lọc nhanh sản phẩm

Tại cột **Thao tác**, quản trị viên có thể dễ dàng quản lý vòng đời của một sản phẩm:

- Bấm "**Thêm Sản Phẩm Mới**" để bổ sung một mặt hàng trang sức vào hệ thống.
- Bấm "**Sửa**" để cập nhật lại thông tin (Thay đổi giá cả khi có khuyến mãi, cập nhật lại số lượng tồn kho sau khi nhập hàng, hoặc thay đổi hình ảnh, mô tả).
- Bấm "**Xóa**" để gỡ bỏ hoàn toàn sản phẩm khỏi hệ thống.

b. Xử lý nghiệp vụ (Back-end) Đối với nghiệp vụ Quản trị Sản phẩm, hệ thống áp dụng kỹ thuật **Entity Framework Core (EF Core)** để thực hiện các thao tác CRUD (Create - Read - Update - Delete) một cách nhanh chóng và đồng bộ hóa đối tượng (Object-Relational Mapping). Việc sử dụng EF Core

giúp giảm thiểu rủi ro sai sót trong quá trình thao tác với cơ sở dữ liệu so với việc viết các câu lệnh SQL thuần.

Dưới đây là ví dụ về phương thức hỗ trợ Thêm mới sản phẩm vào hệ thống. API sẽ tiếp nhận đối tượng Product từ giao diện, tiến hành kiểm tra tính hợp lệ và gọi lệnh `_context.Products.Add()` trước khi lưu thay đổi xuống cơ sở dữ liệu.

Đoạn mã nguồn xử lý thêm Sản phẩm mới (Trích từ tệp `CategoriesController.cs` - Ghi chú: Trong kiến trúc thực tế, đoạn này thường được đặt tại `ProductsController.cs`):

```
[HttpPost]
public async Task<ActionResult<Product>> PostProduct(Product product)
{
    // Sử dụng Entity Framework Core để thêm bản ghi mới
    _context.Products.Add(product);

    // Đồng bộ hóa dữ liệu xuống SQL Server
    await _context.SaveChangesAsync();

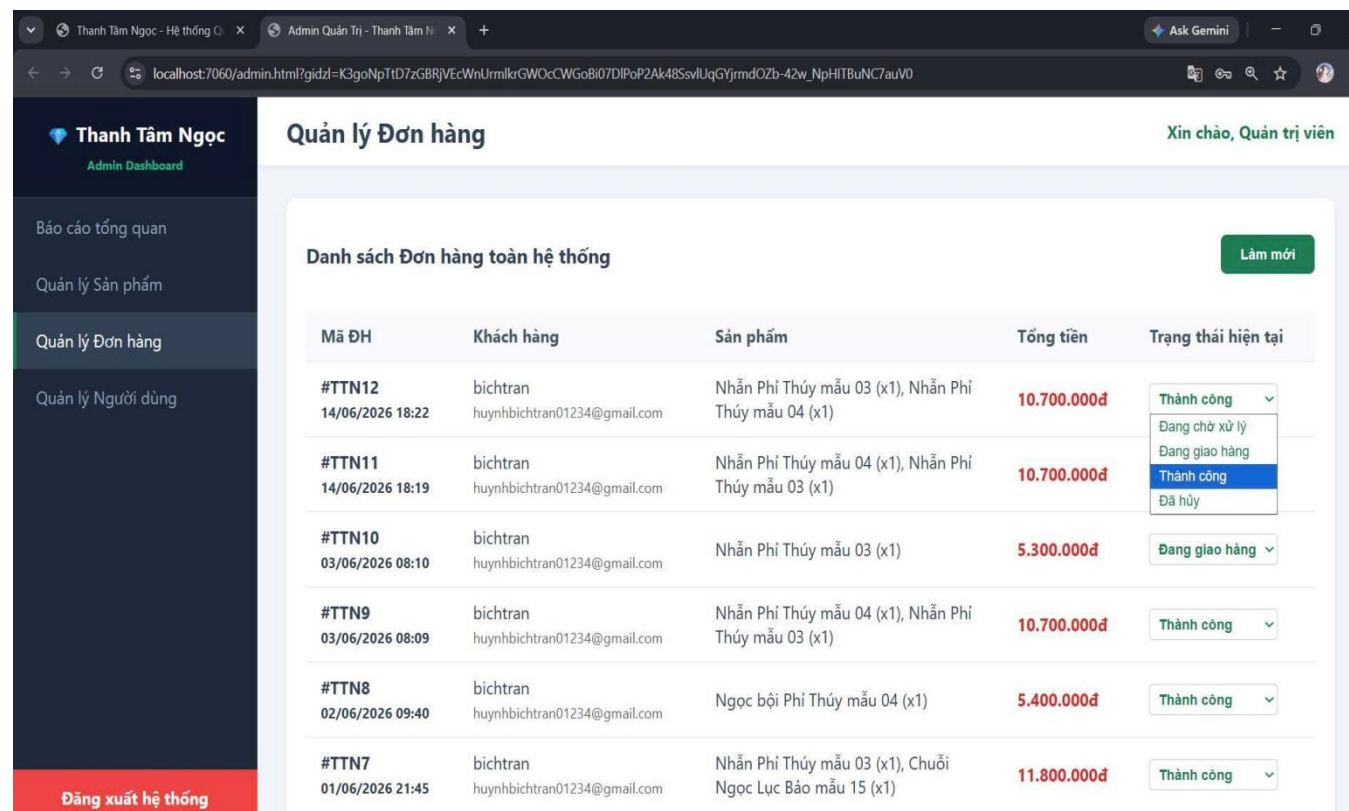
    // Trả về thông tin sản phẩm vừa được tạo thành công
    return Ok(product);
}
```

3.8.3. Chức năng Quản lý Đơn hàng

a. Giao diện người dùng (Front-end)

Đây là phân hệ giúp Quản trị viên theo dõi và điều phối toàn bộ luồng giao dịch của hệ thống. Giao diện hiển thị dưới dạng danh sách tổng hợp, bao gồm các thông tin thiết yếu: Mã đơn hàng (kèm thời gian đặt), Thông tin khách hàng, Chi tiết sản phẩm (đã được gộp chuỗi), và Tổng tiền.

Điểm nhấn về mặt Trải nghiệm người dùng (UX) tại đây là cột **Trạng thái hiện tại**. Thay vì phải chuyển sang một trang chỉnh sửa phức tạp, hệ thống tích hợp trực tiếp thẻ Select (Dropdown menu) ngay trên từng dòng dữ liệu. Quản trị viên chỉ cần nhấp chuột để thay đổi tiến trình của đơn hàng theo thực tế (Đang chờ xử lý, Đang giao hàng, Thành công, Đã hủy). Hệ thống sẽ lập tức cập nhật màu sắc và trạng thái tương ứng.



Hình 3.19. Giao diện danh sách Đơn hàng và thao tác cập nhật Trạng thái

b. Xử lý nghiệp vụ (Back-end)

Quá trình cập nhật trạng thái đơn hàng diễn ra thông qua phương thức gọi API dạng PUT hoặc POST tới máy chủ. Tập **AdminController.cs** sẽ tiếp nhận tham số OrderId (Mã đơn) và Status (Trạng thái mới).

Hệ thống sử dụng đối tượng SqlConnection và SqlCommand để thực thi trực tiếp câu lệnh UPDATE vào bảng Orders trong cơ sở dữ liệu SQL Server. Kỹ thuật truyền tham số (Parameters.AddWithValue) tiếp tục được áp dụng nghiêm ngặt để đảm bảo an toàn dữ liệu. Ngay sau khi lệnh cập nhật trả về số dòng bị ảnh hưởng (ExecuteNonQuery() > 0), API sẽ phản hồi thông báo thành công về cho Client để giao diện tự động đồng bộ.

Đoạn mã nguồn xử lý cập nhật trạng thái Đơn hàng (Trích từ tập AdminController.cs):

```
[HttpPut("orders/{id}/status")]
public IActionResult UpdateOrderStatus(int id, [FromBody] string newStatus)
{
    try
    {
        using (SqlConnection conn = new SqlConnection(_connectionString))
        {
            conn.Open();
            // Cập nhật trạng thái mới cho đơn hàng tương ứng
            string query = "UPDATE Orders SET OrderStatus = @Status WHERE OrderId = @Id";

            using (SqlCommand cmd = new SqlCommand(query, conn))
            {
                // Tham số hóa để bảo mật SQL Injection
                cmd.Parameters.AddWithValue("@Status", newStatus);
                cmd.Parameters.AddWithValue("@Id", id);

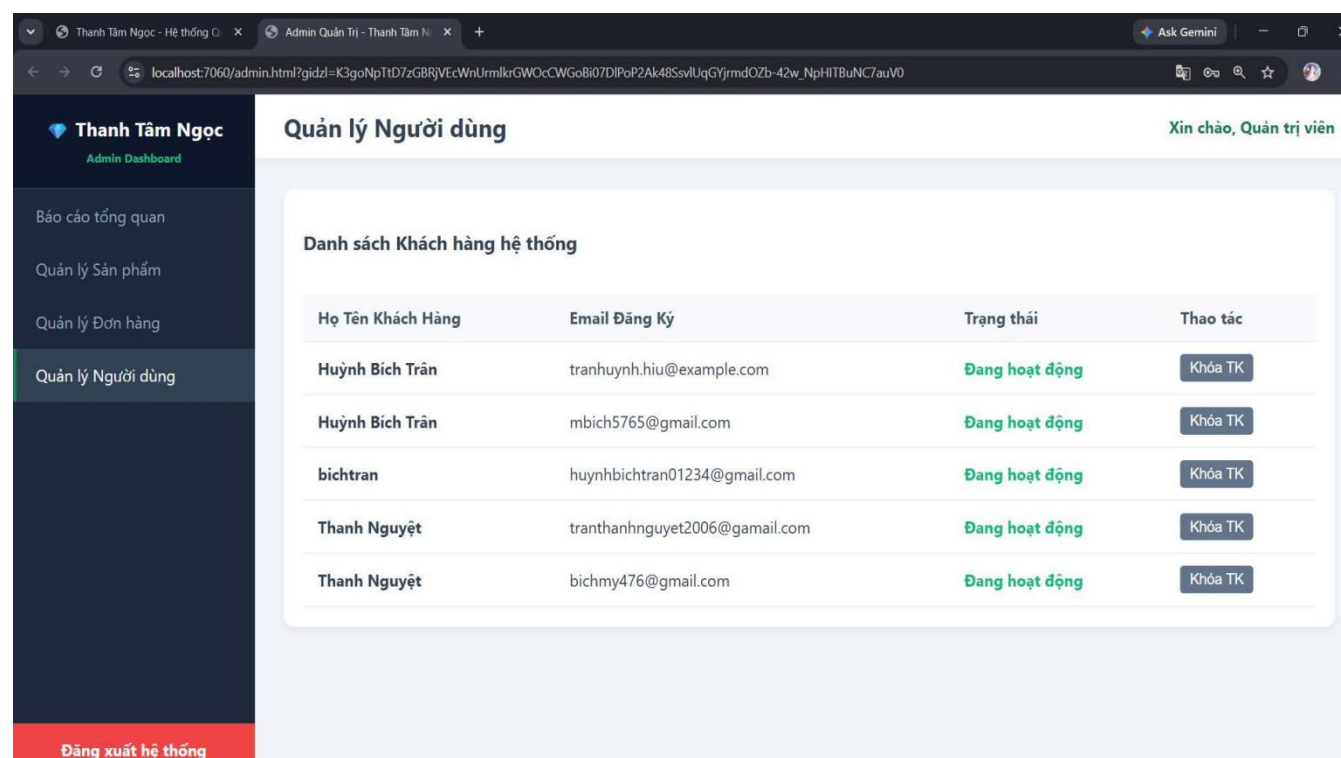
                int rowsAffected = cmd.ExecuteNonQuery();

                if (rowsAffected > 0)
                {
                    return Ok(new { message = "Cập nhật trạng thái đơn hàng thành công!" });
                }
                else
                {
                    return NotFound(new { message = "Lỗi: Không tìm thấy mã đơn hàng trong hệ thống." });
                }
            }
        }
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { message = "Lỗi máy chủ: " + ex.Message });
    }
}
```

3.8.4. Chức năng Quản lý Người dùng

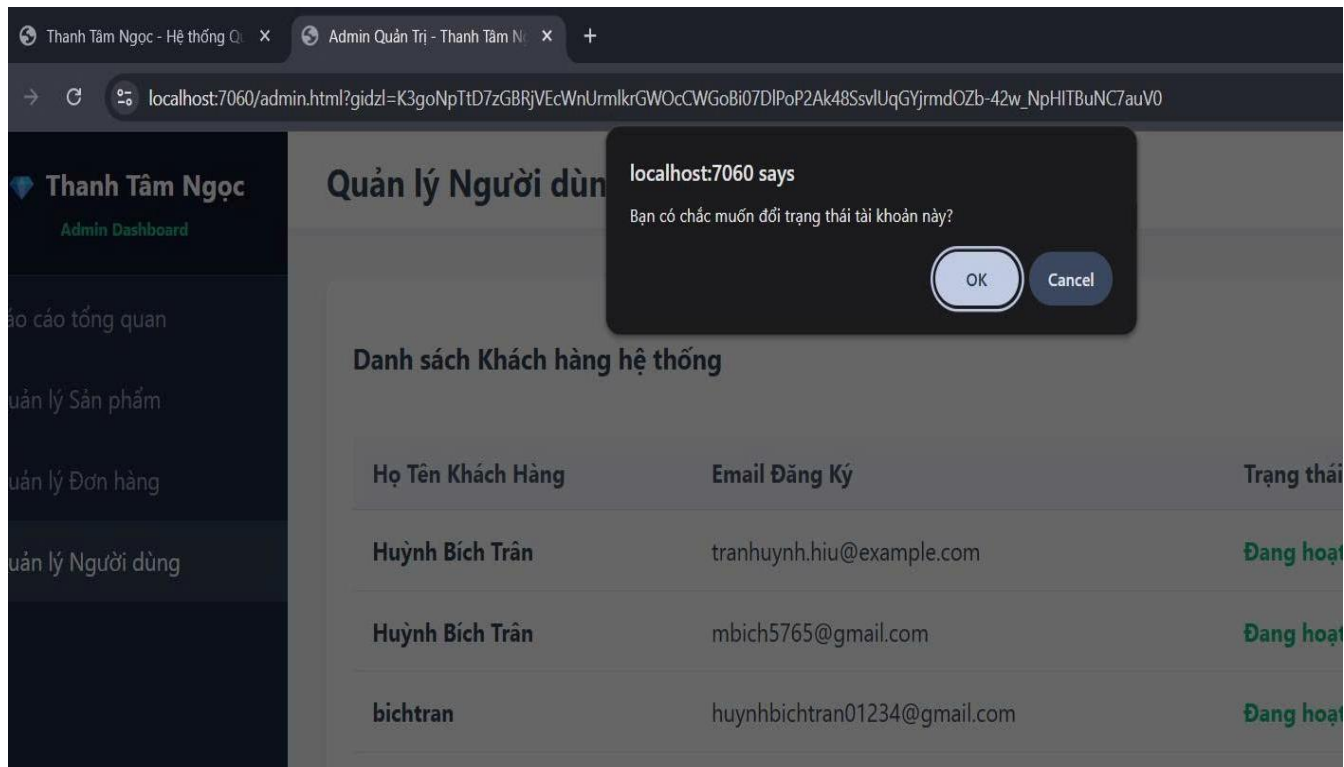
a. Giao diện người dùng (Front-end)

Nhằm duy trì một môi trường thương mại điện tử lành mạnh, Quản trị viên được cấp quyền giám sát toàn bộ danh sách khách hàng đã đăng ký tài khoản trên hệ thống Thanh Tâm Ngọc. Giao diện liệt kê rõ ràng Họ tên, Địa chỉ Email và đặc biệt là Trạng thái hoạt động hiện tại của từng người dùng.



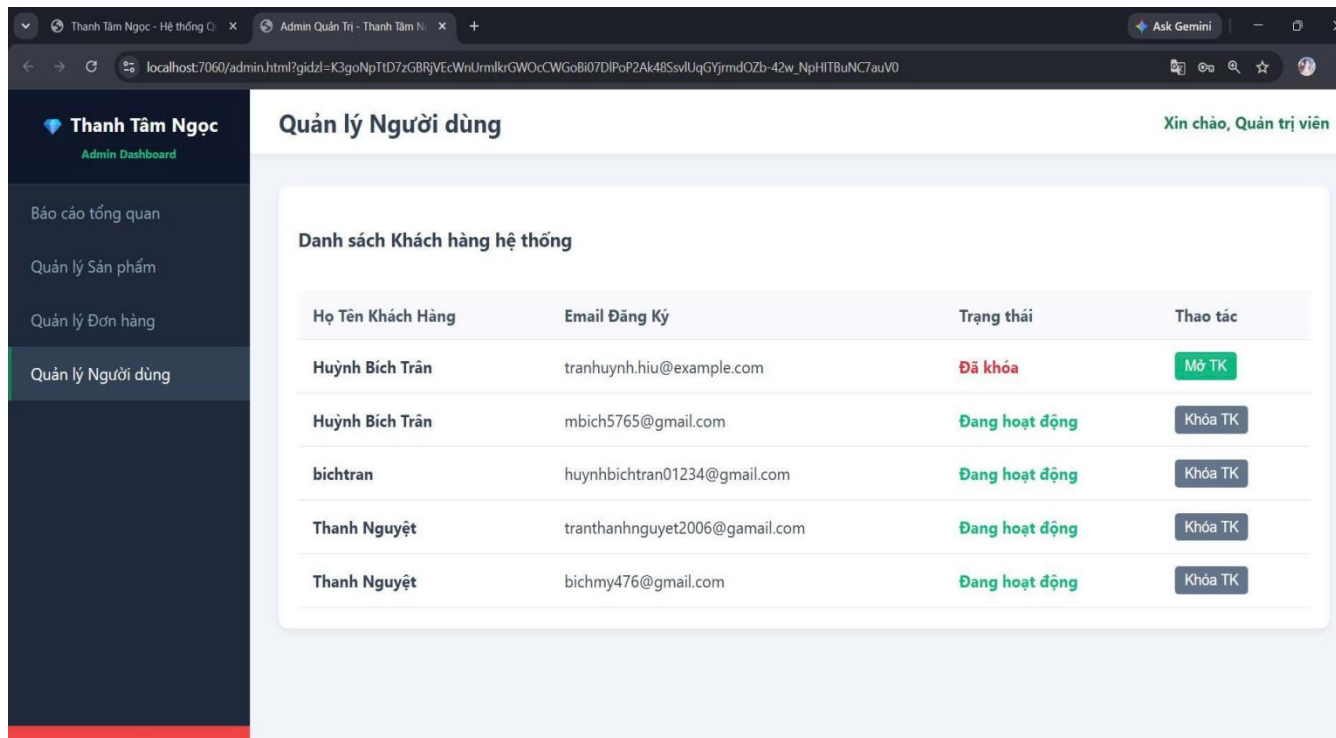
Hình 3.20. Giao diện tổng quan danh sách Khách hàng hệ thống

Trong quá trình vận hành, nếu phát hiện tài khoản có dấu hiệu bất thường (spam đơn hàng ảo, vi phạm chính sách), Quản trị viên có thể can thiệp ngay lập tức bằng cách bấm vào nút "Khóa TK". Để tránh thao tác nhầm lẫn, một hộp thoại xác nhận (Confirm) sẽ xuất hiện yêu cầu Admin chốt lại quyết định.



Hình 3.21. Hộp thoại xác nhận trước khi thay đổi trạng thái tài khoản

Sau khi cập nhật thành công, giao diện sẽ phản hồi trực quan bằng việc đổi trạng thái thành chữ đỏ "Đã khóa" và nút thao tác biến thành màu xanh "Mở TK", cho phép Admin dễ dàng hoàn tác (khôi phục tài khoản) bất cứ lúc nào.



Hình 3.22. Giao diện hiển thị sau khi tài khoản đã bị khóa thành công

b. Xử lý nghiệp vụ (Back-end)

Về mặt cơ sở dữ liệu, bảng Users được thiết kế thêm một trường dữ liệu TrạngThai kiểu số nguyên (INT) với giá trị mặc định là 1 (Đang hoạt động). Khi tài khoản bị khóa, giá trị này sẽ chuyển thành 0.

Nghiệp vụ này được thực thi tại tệp **AdminController.cs** thông qua API [HttpPut("users/{email}/toggle-status")]. Nhằm tối ưu hóa hiệu năng, hệ thống không dùng lệnh Update gán giá trị cứng, mà sử dụng toán tử Bitwise XOR (^) hoặc phép trừ logic trong câu lệnh SQL để đảo ngược trạng thái (1 thành 0, và ngược lại) chỉ bằng một thao tác duy nhất.

Đoạn mã nguồn xử lý Khóa/Mở tài khoản khách hàng (Trích từ tệp AdminController.cs):

```
[HttpPut("users/{email}/toggle-status")]
public IActionResult ToggleUserStatus(string email)
{
    try
    {
        using (SqlConnection conn = new SqlConnection(_connectionString))
        {
            conn.Open();
            // Câu lệnh SQL đảo ngược trạng thái: Nếu 1 thì thành 0, nếu 0 thì thành 1
            string query = "UPDATE Users SET TrangThai = 1 - TrangThai WHERE Email = @Email";

            using (SqlCommand cmd = new SqlCommand(query, conn))
            {
                // Truyền tham số an toàn chống SQL Injection
                cmd.Parameters.AddWithValue("@Email", email);

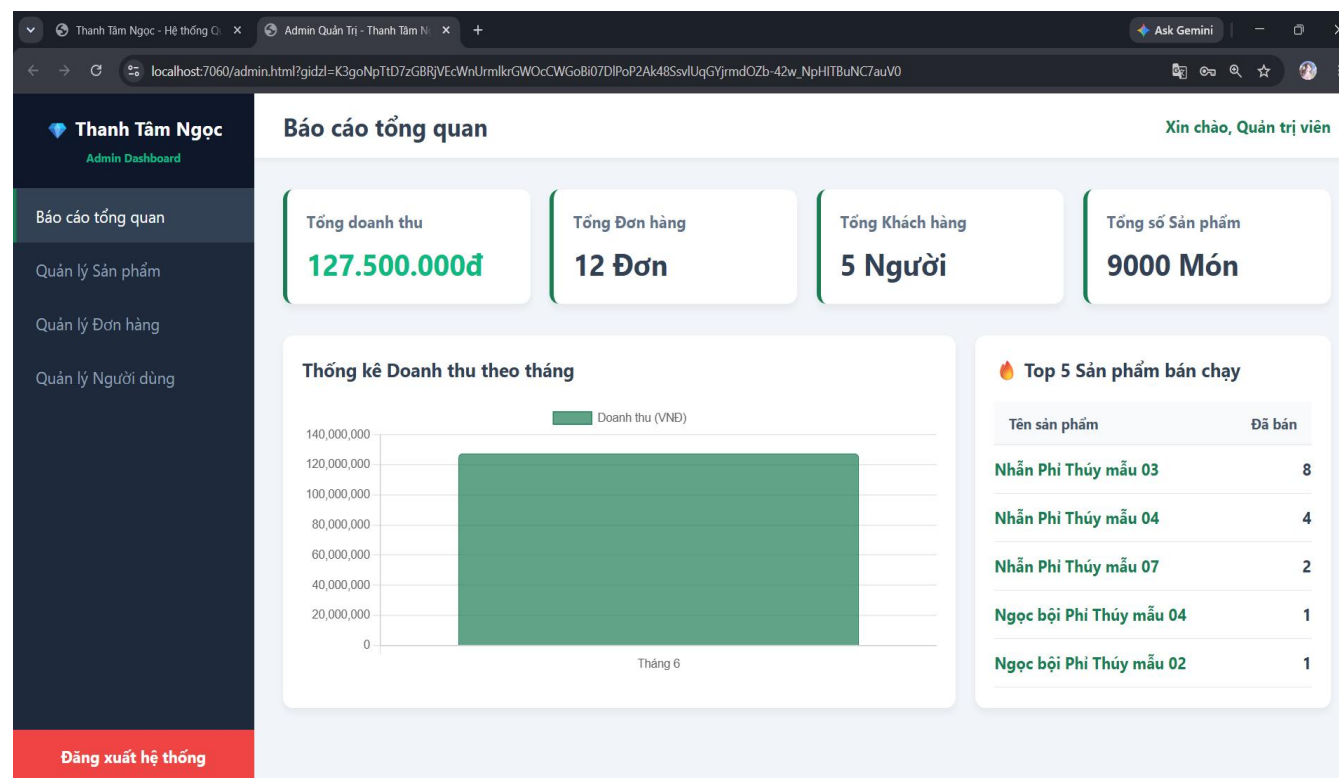
                int rowsAffected = cmd.ExecuteNonQuery();

                if (rowsAffected > 0)
                {
                    return Ok(new { message = "Cập nhật trạng thái tài khoản thành công!" });
                }
                else
                {
                    return NotFound(new { message = "Không tìm thấy khách hàng này trong hệ thống." });
                }
            }
        }
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { message = "Lỗi Server: " + ex.Message });
    }
}
```

3.8.5. Chức năng Báo cáo Thống kê Tổng quan (Dashboard)

a. Giao diện người dùng (Front-end)

Ngay sau khi đăng nhập thành công vào trang Quản trị, hệ thống sẽ tự động hiển thị màn hình Báo cáo tổng quan (Dashboard). Đây là trung tâm kiểm soát, cung cấp cho Quản trị viên cái nhìn toàn cảnh về tình hình kinh doanh của cửa hàng thông qua các chỉ số theo thời gian thực.

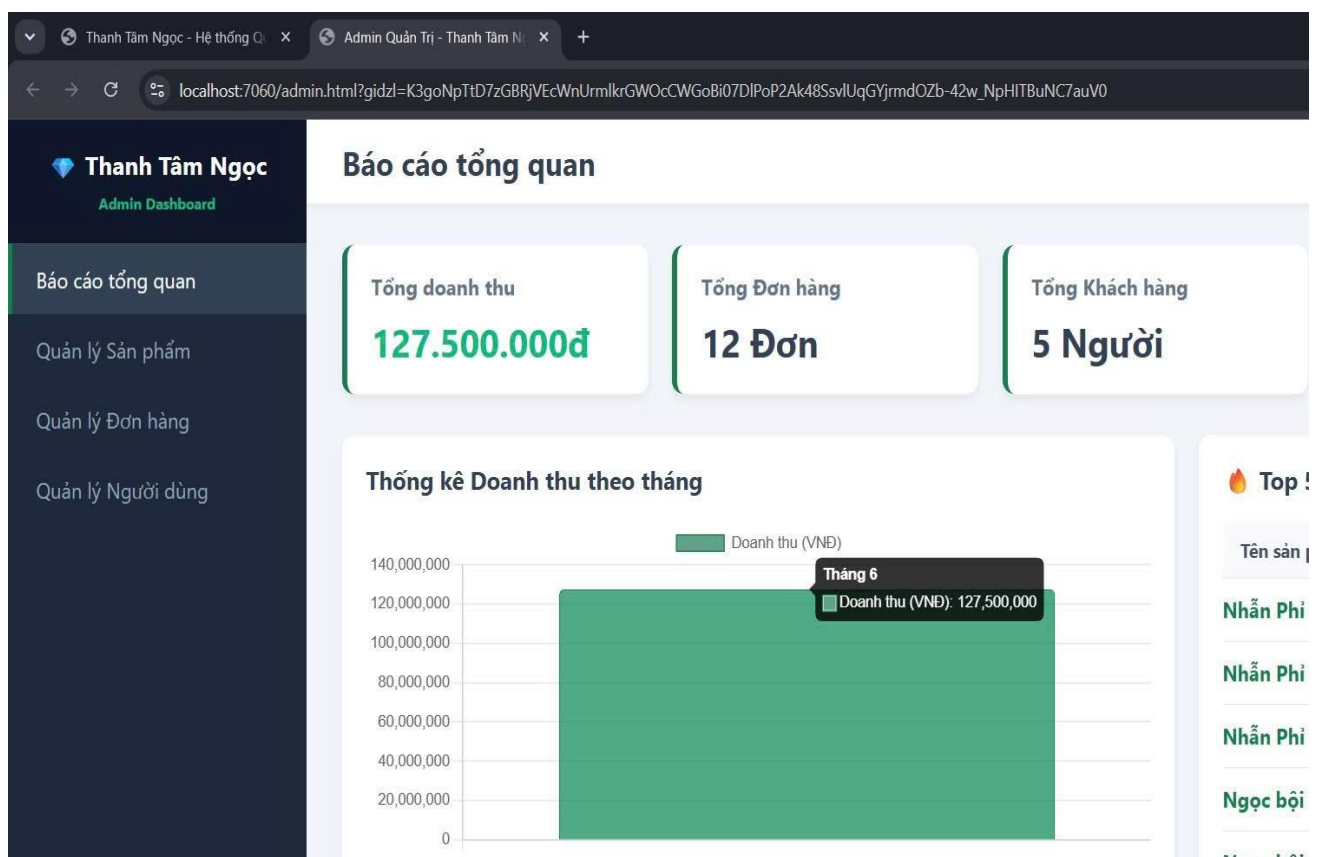


Hình 3.23. Giao diện Báo cáo tổng quan (Dashboard) của hệ thống

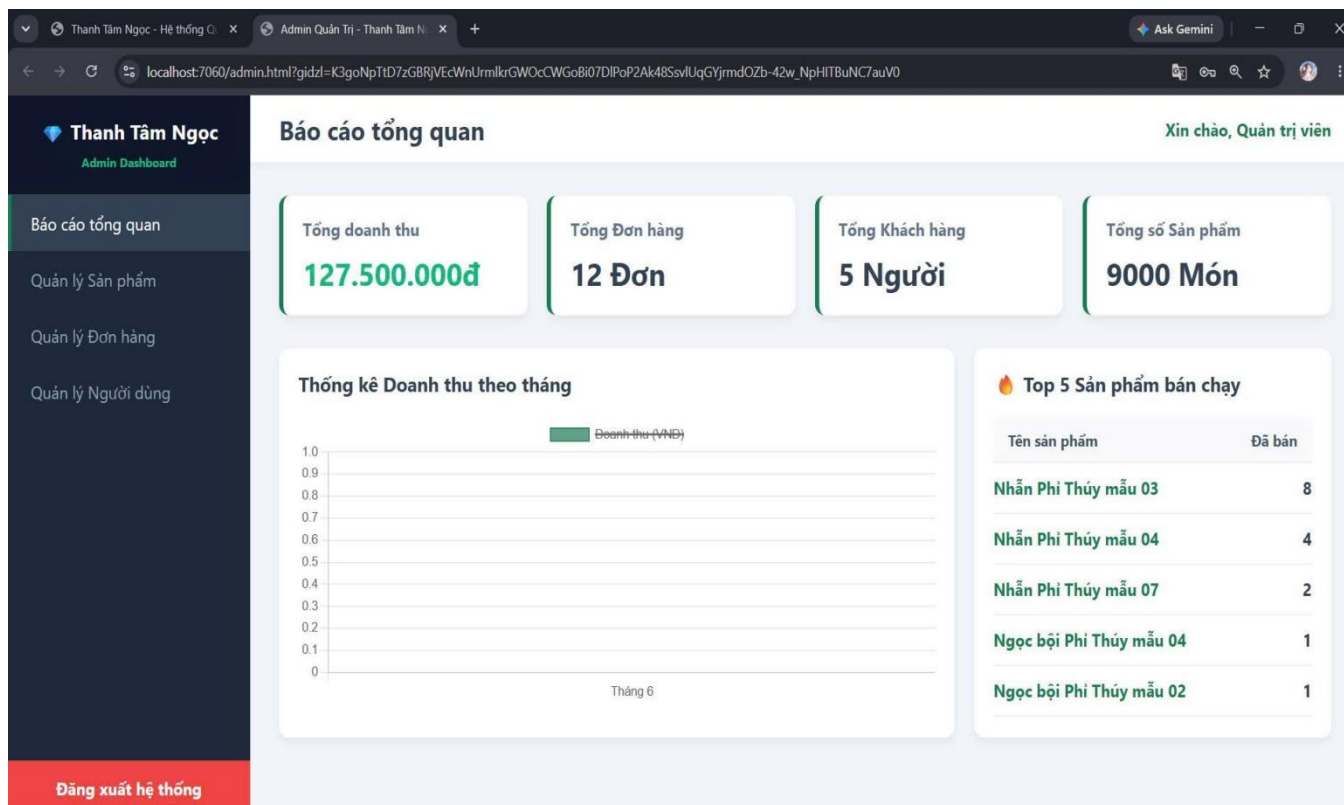
Giao diện được thiết kế trực quan, chia thành các phân khu rõ ràng:

- **Thẻ thông số nhanh (Summary Cards):** Hiển thị 4 chỉ số cốt lõi: Tổng doanh thu đã đạt được, Tổng số đơn hàng thành công, Tổng số khách hàng đăng ký và Tổng lượng sản phẩm đang lưu trữ trong kho.

- **Bảng xếp hạng (Leaderboard):** Vinh danh "Top 5 Sản phẩm bán chạy", giúp định hướng chiến lược nhập hàng và tiếp thị trong tương lai.
- **Biểu đồ Doanh thu (Revenue Chart):** Nhằm tăng tính tương tác (Interactive UI), hệ thống tích hợp thư viện Chart.js để vẽ biểu đồ cột. Khi quản trị viên rê chuột (hover) vào các cột, một tooltip nhỏ sẽ xuất hiện hiển thị con số doanh thu chính xác của tháng đó. Đặc biệt, khi nhấp chuột (click) vào chú thích "Doanh thu (VNĐ)" ở cạnh trên, biểu đồ sẽ tạm thời ẩn đi để người dùng tập trung vào các số liệu khác.



Hình 3.24. Hiệu ứng Tooltip hiển thị số liệu chi tiết khi rê chuột vào biểu đồ



Hình 3.25. Tính năng ẩn/hiện cột dữ liệu động trên biểu đồ Chart.js

b. Xử lý nghiệp vụ (Back-end)

Dữ liệu cung cấp cho bảng điều khiển được tổng hợp từ nhiều nguồn thông qua API [HttpGet("chart-data")] nằm trong tệp AdminController.cs.

Để đảm bảo hiệu năng khi tính toán trên lượng dữ liệu lớn, hệ thống sử dụng kết nối trực tiếp SqlConnection và thực thi các câu lệnh SQL Server tối ưu. Đặc biệt, để lấy ra danh sách 5 sản phẩm bán chạy nhất, mã nguồn sử dụng mệnh đề SELECT TOP 5 kết hợp với lệnh JOIN để nối bảng OrderDetails và bảng Orders (chỉ tính những đơn hàng không bị hủy), sau đó dùng hàm SUM(Quantity) và GROUP BY để nhóm và đếm tổng số lượng bán ra của từng mặt hàng. Toàn bộ dữ liệu sau khi tổng hợp sẽ được đóng gói thành một cấu trúc JSON duy nhất để trả về cho Front-end vẽ biểu đồ.

Đoạn mã nguồn xử lý trích xuất dữ liệu Báo cáo (Trích từ tệp AdminController.cs):

```
[HttpGet("chart-data")]
public IActionResult GetChartData()
{
    try
    {
        var chartLabels = new List<string>();
        var chartValues = new List<decimal>();
        var topProducts = new List<object>();

        using (SqlConnection conn = new SqlConnection(_connectionString))
        {
            conn.Open();
            // 1. Gôm dữ liệu về Biểu đồ: Tổng doanh thu theo từng tháng
            string queryChart = @"
                SELECT 'Tháng ' + CAST(MONTH(OrderDate) AS VARCHAR) AS MonthLabel,
                       SUM(TotalAmount) AS Revenue
                FROM Orders
                WHERE OrderStatus <> N'Đã hủy'
                GROUP BY MONTH(OrderDate)
                ORDER BY MONTH(OrderDate)";

            using (SqlCommand cmd = new SqlCommand(queryChart, conn))
            using (SqlDataReader reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    chartLabels.Add(reader["MonthLabel"].ToString());
                    chartValues.Add(Convert.ToDecimal(reader["Revenue"]));
                }
            }
        }

        // 2. Gôm dữ liệu Top 5 sản phẩm bán chạy nhất
        string queryTop = @"
            SELECT TOP 5 ProductName, SUM(Quantity) AS TotalSold
            FROM OrderDetails od
            JOIN Orders o ON od.OrderId = o.OrderId
            WHERE o.OrderStatus <> N'Đã hủy'
            GROUP BY ProductName
            ORDER BY TotalSold DESC";

        using (SqlCommand cmd = new SqlCommand(queryTop, conn))
        using (SqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                topProducts.Add(new
                {
                    name = reader["ProductName"].ToString(),
                    sold = Convert.ToInt32(reader["TotalSold"])
                });
            }
        }

        // Gói tất cả dữ liệu thành JSON và gửi về cho Chart.js xử lý
        return Ok(new { labels = chartLabels, values = chartValues, top = topProducts });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { message = ex.Message });
    }
}
```

3.9. XÂY DỰNG MÁY CHỦ BACK-END (REST API) VÀ CƠ SỞ DỮ LIỆU

Phần hệ máy chủ của hệ thống Thanh Tâm Ngọc được xây dựng bằng kiến trúc RESTful API trên nền tảng C# ASP.NET Core, chia thành các phân hệ xử lý nghiệp vụ độc lập theo chuẩn.

Backend (REST API)

- **Auth & Roles:** phân quyền user (khách hàng) và admin.
- **Products API:** CRUD sản phẩm; phân trang & tìm kiếm.
- **Cart API:** thêm/xóa/cập nhật sản phẩm trong giỏ hàng; tính tổng giá trị.
- **Orders API:** tạo đơn hàng, cập nhật trạng thái, truy xuất lịch sử.
- **Payment API:** xử lý thanh toán online (mock hoặc tích hợp cổng thật).
- **Reports API:** doanh thu, sản phẩm bán chạy, dữ liệu theo ngày/tháng.

***Hình 3.26.** Danh sách yêu cầu thiết kế Backend REST API của hệ thống*

a. Auth & Roles (Xác thực và Phân quyền)

Hệ thống triển khai cơ chế phân quyền rõ rệt giữa hai nhóm đối tượng: Khách hàng (Customer) và Quản trị viên (Admin). Toàn bộ quy trình đăng nhập, đăng ký và cấp quyền truy cập đều được mã hóa và bảo vệ. Mọi thao tác truy cập vào các API nhạy cảm đều yêu cầu token xác thực hợp lệ thông qua thuộc tính [Authorize].

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(keyBytes)
        };
    });

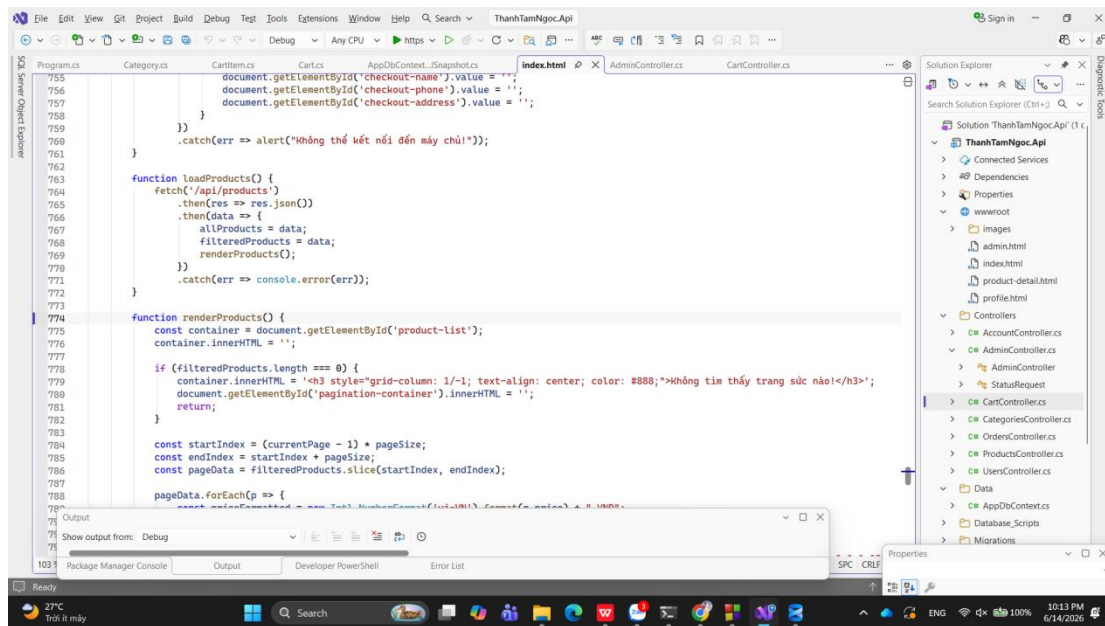
builder.Services.AddControllers().AddJsonOptions(x =>
    x.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles);

builder.Services.AddOpenApi();
```

Hình 3.27. Mã nguồn cấu hình xác thực và phân quyền (Authorization)

b. Products API (Quản lý và Truy xuất Sản phẩm)

Cung cấp bộ giao tiếp cho phép thực hiện các thao tác CRUD (Thêm, Đọc, Sửa, Xóa) đối với kho trang sức. Khách hàng sử dụng Fetch API từ giao diện để gọi xuống máy chủ lấy danh sách sản phẩm. Backend hỗ trợ tìm kiếm động theo tên, lọc theo danh mục hoặc phân khúc giá cả, đồng thời hỗ trợ phân trang để tối ưu hiệu năng.



Hình 3.28. Client gọi Fetch API đến endpoint /api/products để nạp dữ liệu

```
[HttpGet]
0 references
public IActionResult GetProducts([FromQuery] string search = "", [FromQuery] int? categoryId = null, [FromQuery] decimal? minPrice = null, [FromQuery] decimal? maxPrice = null)
{
    try
    {
        List<object> products = new List<object>();

        using (SqlConnection connection = new SqlConnection(_connectionString))
        {
            connection.Open();

            string sql = "SELECT * FROM Products WHERE 1=1";

            if (!string.IsNullOrEmpty(search))
            {
                sql += " AND Name LIKE @Search";
            }
            if (categoryId.HasValue)
            {
                sql += " AND CategoryId = @CategoryId";
            }
            if (minPrice.HasValue)
            {
                sql += " AND Price >= @MinPrice";
            }
            if (maxPrice.HasValue)
            {
                sql += " AND Price <= @MaxPrice";
            }

            using (SqlCommand command = new SqlCommand(sql, connection))
            {
                if (!string.IsNullOrEmpty(search))
                {
                    command.Parameters.AddWithValue("@Search", "%" + search + "%");
                }
                if (categoryId.HasValue)
                {

```

```
using (SqlCommand command = new SqlCommand(sql, connection))
{
    if (!string.IsNullOrEmpty(search))
    {
        command.Parameters.AddWithValue("@Search", "%" + search + "%");
    }
    if (categoryId.HasValue)
    {
        command.Parameters.AddWithValue("@CategoryId", categoryId.Value);
    }
    if (minPrice.HasValue)
    {
        command.Parameters.AddWithValue("@MinPrice", minPrice.Value);
    }
    if (maxPrice.HasValue)
    {
        command.Parameters.AddWithValue("@MaxPrice", maxPrice.Value);
    }

    using (SqlDataReader reader = command.ExecuteReader())
    {
        while (reader.Read())
        {
            products.Add(new
            {
                id = reader["Id"],
                name = reader["Name"],
                price = reader["Price"],
                imageUrl = reader["ImageUrl"] != DBNull.Value ? reader["ImageUrl"].ToString() : "",
                gemType = reader["GemType"] != DBNull.Value ? reader["GemType"].ToString() : "",
                description = reader["Description"] != DBNull.Value ? reader["Description"].ToString() : ""
            });
        }
    }
}

return Ok(products);
}
catch (Exception ex)
{
    return StatusCode(500, "Internal server error: " + ex.Message);
}
```

Hình 3.29. Mã nguồn C# xử lý truy vấn động tìm kiếm sản phẩm

c. Cart API (Quản lý Giỏ hàng)

Mỗi khách hàng được hệ thống định danh quản lý một Giỏ hàng riêng biệt. API hỗ trợ các thao tác linh hoạt: Thêm sản phẩm mới, tự động cộng dồn số lượng nếu sản phẩm đã tồn tại, và xóa sản phẩm. Hệ thống sử dụng Entity Framework Core để tính toán tổng giá trị tạm tính một cách toàn vẹn.

```
[HttpPost("add")]
0 references
public async Task<ActionResult> AddToCart([FromBody] CartItemRequest request)
{
    var userId = GetCurrentUserId();

    var cart = await _context.Carts.Include(c => c.CartItems).FirstOrDefaultAsync(c => c.UserId == userId);
    if (cart == null)
    {
        cart = new Cart { UserId = userId };
        _context.Carts.Add(cart);
        await _context.SaveChangesAsync();
    }

    var existingItem = cart.CartItems?.FirstOrDefault(ci => ci.ProductId == request.ProductId);
    if (existingItem != null)
    {
        existingItem.Quantity += request.Quantity;
    }
    else
    {
        var newItem = new CartItem
        {
            CartId = cart.Id,
            ProductId = request.ProductId,
            Quantity = request.Quantity
        };
        _context.CartItems.Add(newItem);
    }

    await _context.SaveChangesAsync();
    return Ok(new { message = "Đã bỏ ngọc quý vào giỏ thành công rực rỡ!" });
}
```

Hình 3.30. Mã nguồn kiểm tra và xử lý thêm sản phẩm vào Giỏ hàng

d. Orders API (Quản lý Đơn hàng)

Phụ trách quy trình giao dịch cốt lõi. Khi khách hàng xác nhận mua, API sẽ khởi tạo đơn hàng mới (INSERT INTO Orders), lưu chi tiết từng mặt hàng (INSERT INTO OrderDetails) và gán trạng thái ban đầu là "Đang chờ xử lý".

[HttpPost("checkout")]

0 references

public IActionResult Checkout([FromBody] CheckoutRequest request)

```
{
    try
    {
        using (SqlConnection conn = new SqlConnection(_connectionString))
        {
            conn.Open();
            string orderQuery = @"
                INSERT INTO Orders (UserEmail, ShippingAddress, TotalAmount, OrderStatus)
                OUTPUT INSERTED.OrderId
                VALUES (@Email, @Address, @TotalAmount, N'Thành công');

            int orderId = 0;
            using (SqlCommand cmd = new SqlCommand(orderQuery, conn))
            {
                cmd.Parameters.AddWithValue("@Email", request.Email);
                cmd.Parameters.AddWithValue("@Address", request.Address);
                cmd.Parameters.AddWithValue("@TotalAmount", request.TotalAmount);
                orderId = (int)cmd.ExecuteScalar();
            }

            foreach (var item in request.Items)
            {
                string detailQuery = @"
                    INSERT INTO OrderDetails (OrderId, ProductName, Quantity, UnitPrice)
                    VALUES (@OrderId, @ProductName, @Quantity, @UnitPrice);

                using (SqlCommand detailCmd = new SqlCommand(detailQuery, conn))
                {
                    detailCmd.Parameters.AddWithValue("@OrderId", orderId);
                    detailCmd.Parameters.AddWithValue("@ProductName", item.Name);
                    detailCmd.Parameters.AddWithValue("@Quantity", item.Quantity);
                    detailCmd.Parameters.AddWithValue("@UnitPrice", item.Price);
                    detailCmd.ExecuteNonQuery();
                }
            }
        }

        return Ok(new { message = "Thành công" });
    }

    string detailQuery = @"
        INSERT INTO OrderDetails (OrderId, ProductName, Quantity, UnitPrice)
        VALUES (@OrderId, @ProductName, @Quantity, @UnitPrice);

    using (SqlCommand detailCmd = new SqlCommand(detailQuery, conn))
    {
        detailCmd.Parameters.AddWithValue("@OrderId", orderId);
        detailCmd.Parameters.AddWithValue("@ProductName", item.Name);
        detailCmd.Parameters.AddWithValue("@Quantity", item.Quantity);
        detailCmd.Parameters.AddWithValue("@UnitPrice", item.Price);
        detailCmd.ExecuteNonQuery();
    }
}

return Ok(new { message = "Thành công" });
}
catch (Exception ex)
{
    return StatusCode(500, new { message = "Lỗi hệ thống: " + ex.Message });
}
}
```

Hình 3.31. Mã nguồn xử lý tạo lập hóa đơn và chi tiết hóa đơn

e. Payment API (Thanh toán trực tuyến)

Hệ thống xử lý thanh toán đa luồng. Đối với thanh toán online (VNPay/MoMo), hệ thống tích hợp cơ chế giả lập hiển thị mã VietQR động trực tiếp tại giao diện, giúp khách hàng quét mã dễ dàng trước khi xác nhận lưu đơn hàng xuống cơ sở dữ liệu.

```

<div id="payment-gateway-modal" style="display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: white; padding: 40px; border-radius: 16px; width: 400px; text-align: center; margin: 0 auto;">
    <h2 style="color: #1c7c54; margin-top: 0; margin-bottom: 10px;">Cổng Thanh Toán</h2>
    <p style="color: #555; margin-bottom: 20px;">Vui lòng quét mã QR để thanh toán đơn hàng</p>
    
    <p style="font-size: 1.2em; font-weight: bold; color: #d32f2f; margin-bottom: 25px;">Số tiền: <span id="qr-total">0 VNĐ</span></p>
    <div style="display: flex; gap: 10px;">
        <button onclick="cancelPayment()" style="flex: 1; padding: 12px; background: #eee; color: #333; border: none; font-weight: bold; cursor: pointer; border-radius: 8px; transition: 0.3s;">HỦY BỎ</button>
        <button onclick="confirmPayment()" style="flex: 2; padding: 12px; background: #1c7c54; color: white; border: none; font-weight: bold; cursor: pointer; border-radius: 8px; transition: 0.3s;">ĐÃ THANH TOÁN</button>
    </div>
</div>
</div>

<div id="receipt-modal" style="display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: white; padding: 40px; border-radius: 16px; width: 450px; text-align: center; margin: 0 auto;">
    <div style="font-size: 50px; color: #1c7c54; margin-bottom: 10px;">✓</div>
    <h2 style="color: #1c7c54; margin-top: 0;">Đặt hàng thành công</h2>
    <p style="color: #666; margin-bottom: 20px;">Cảm ơn quý khách đã mua sắm tại Thanh Tâm Ngọc.</p>
    <div style="background: #f4fcf7; padding: 20px; border-radius: 8px; text-align: left; margin-bottom: 10px;">
        <p style="margin: 0 0 10px 0;"><strong>Khách hàng:</strong> <span id="receipt-name"></span></p>
        <p style="margin: 0 0 10px 0;"><strong>Số điện thoại:</strong> <span id="receipt-phone"></span></p>
        <p style="margin: 0 0 10px 0;"><strong>Giao đến:</strong> <span id="receipt-address"></span></p>
        <hr style="border: 0; border-top: 1px solid #ccc; margin: 15px 0;" />
        <p style="margin: 0; font-size: 1.2em;"><strong>Tổng thanh toán:</strong> <span id="receipt-total"></span></p>
    </div>
    <button onclick="closeReceiptModal()" style="width: 100%; padding: 14px; background: #1c7c54; color: white; border: none; font-weight: bold; cursor: pointer; border-radius: 8px; transition: 0.3s;">TIẾP TỤC MUA SẮM</button>
</div>
</div>

```

Hình 3.32. Logic xử lý hiển thị cổng thanh toán quét mã QR

f. Reports API (Báo cáo Thống kê)

Phục vụ riêng cho Admin Dashboard. API thực thi các câu lệnh SQL nâng cao chứa hàm tập hợp (Aggregate Functions) để tính tổng doanh thu và sử dụng lệnh TOP 5 kết hợp GROUP BY để trích xuất danh sách các trang sức bán chạy nhất theo thời gian thực.

```
// 2. Gom dữ liệu Top 5 sản phẩm bán chạy nhất
string queryTop = @"
    SELECT TOP 5 ProductName, SUM(Quantity) AS TotalSold
    FROM OrderDetails od
    JOIN Orders o ON od.OrderId = o.OrderId
    WHERE o.OrderStatus <> N'Đã hủy'
    GROUP BY ProductName
    ORDER BY TotalSold DESC";
```

Hình 3.33. Câu lệnh SQL nâng cao xử lý thống kê top sản phẩm

g. Quản lý Cơ sở dữ liệu (Database)

Toàn bộ cấu trúc bảng (Tables) và ràng buộc toàn vẹn được thiết kế bằng SQL Server. Để thuận tiện cho việc bảo trì, các tệp lệnh (Script) được chia nhỏ thành các tệp tin chuyên biệt thay vì gộp chung.

```

1  -- ĐỒ ÁN: HỆ THỐNG QUẢN LÝ VÀ PHÂN PHỐI TRANG SỨC THANH TÂM NGỌC
2  -- NGƯỜI THỰC HIỆN: HUỖNH BÍCH TRẦN
3
4  -- 1. TẠO CƠ SỞ DỮ LIỆU
5  CREATE DATABASE ThanhTamNgocDb;
6  GO
7
8  USE ThanhTamNgocDb;
9  GO
10
11 -- 2. TẠO BẢNG TÀI KHOẢN (KHÁCH HÀNG & ADMIN)
12 CREATE TABLE Users (
13     Email VARCHAR(100) PRIMARY KEY,
14     Password VARCHAR(255) NOT NULL,
15     FullName NVARCHAR(100) NOT NULL,
16     Phone VARCHAR(15),
17     Address NVARCHAR(255),
18     Role VARCHAR(20) DEFAULT 'Customer' -- 'Admin' hoặc 'Customer'
19 );
20 GO
21
22 -- 3. TẠO BẢNG DANH MỤC TRANG SỨC
23 CREATE TABLE Categories (
24     CategoryId INT IDENTITY(1,1) PRIMARY KEY,
25     CategoryName NVARCHAR(100) NOT NULL
26 );
27 GO
  
```

Hình 3.34. Mã lệnh T-SQL tạo lập cơ sở dữ liệu và các bảng cốt lõi

Name	Date modified	Type	Size
00_Database_DoAn_ThanhTamNgoc.sql	6/1/2026 8:38 PM	Microsoft SQL Ser...	4 KB
01_QuanLyTaiKhoan.sql	6/3/2026 1:47 PM	Microsoft SQL Ser...	2 KB
02_QuanLyTrangSuc.sql	6/3/2026 10:51 AM	Microsoft SQL Ser...	2 KB
03_QuanLyDanhMuc_TimKiem.sql	5/12/2026 10:56 PM	Microsoft SQL Ser...	1 KB
04_QuanLyDonHang.sql	6/1/2026 11:05 PM	Microsoft SQL Ser...	1 KB

Hình 3.35. Cấu trúc các tệp tin lưu trữ mã lệnh SQL Server của dự án

CHƯƠNG 4: TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

4.1. ĐÁNH GIÁ KẾT QUẢ ĐẠT ĐƯỢC

Dựa vào yêu cầu đặt ra ban đầu của đề án, em đã nghiên cứu và xây dựng thử nghiệm thành công hệ thống website thương mại điện tử phân phối trang sức Thanh Tâm Ngọc, đáp ứng được hầu hết các tiêu chí chuyên môn:

Những mặt đạt được:

- Về mặt giao diện (Front-end):** Giao diện được thiết kế hiện đại, thân thiện, tương thích với nhiều kích thước màn hình (Responsive Design) giúp trải nghiệm trên máy tính hay thiết bị di động đều mượt mà.
- Về mặt kiến trúc (Back-end):** Xây dựng thành công mô hình máy chủ RESTful API bằng ngôn ngữ C# (ASP.NET Core) và cơ sở dữ liệu SQL Server, đảm bảo tính tách biệt, dễ bảo trì và mở rộng.
- Trải nghiệm người dùng:** Chức năng tìm kiếm và lọc sản phẩm hoạt động trơn tru; hệ thống giỏ hàng tính toán chính xác; luồng đặt hàng và thanh toán trực quan.
- Hệ thống bảo mật:** Tích hợp thành công cơ chế xác thực người dùng bằng JWT Token, phân quyền rõ rệt giữa Khách hàng (Customer) và Quản trị viên (Admin).
- Quản trị hệ thống:** Xây dựng trang Admin Dashboard hoàn chỉnh với đầy đủ các nghiệp vụ (Thêm/Sửa/Xóa sản phẩm, duyệt đơn hàng, khóa tài khoản) và biểu đồ báo cáo thống kê trực quan bằng Chart.js.

Những hạn chế cần khắc phục:

- **Cổng thanh toán:** Tính năng thanh toán quét mã VietQR hiện tại chủ yếu mang tính chất mô phỏng giao diện tĩnh (Mock), chưa tích hợp Webhook để nhận tín hiệu phản hồi tự động từ ngân hàng thực tế.
- **Công nghệ Front-end:** Giao diện vẫn đang sử dụng HTML/JS thuần (Vanilla JavaScript). Việc quản lý trạng thái (State management) trên quy mô lớn sẽ chưa thực sự tối ưu bằng các Framework hiện đại.
- **Triển khai (Deployment):** Website và hệ thống API hiện tại mới chỉ hoạt động ổn định ở môi trường máy chủ cục bộ (Localhost), chưa được đóng gói và đưa lên các máy chủ đám mây thực tế.

4.2. HƯỚNG PHÁT TRIỂN

Nhằm hoàn thiện và nâng cao giá trị thực tiễn cho dự án trong tương lai, hệ thống Thanh Tâm Ngọc có thể tiếp tục được phát triển theo các định hướng sau:

- **Nâng cấp công nghệ Front-end:** Chuyển đổi mã nguồn giao diện sang các Framework hiện đại như ReactJS hoặc VueJS để tối ưu hóa tốc độ tải trang và trải nghiệm tương tác (Single Page Application).
- **Hoàn thiện thanh toán trực tuyến:** Tích hợp chính thức các cổng thanh toán điện tử như VNPay, MoMo, hoặc kết nối API ngân hàng để xử lý giao dịch tự động.
- **Tích hợp Đăng nhập Mạng xã hội:** Bổ sung tính năng đăng nhập nhanh thông qua tài khoản Google, Facebook (OAuth 2.0) để mang lại sự tiện lợi cho khách hàng.
- **Hệ thống Gợi ý (Recommendation System):** Xây dựng thuật toán phân tích lịch sử mua hàng để tự động đề xuất các mẫu trang sức (Ngọc Bội, Nhẫn, Dây chuyền) phù hợp với sở thích của từng cá nhân.
- **Triển khai lên Đám mây (Cloud):** Đưa cơ sở dữ liệu và mã nguồn API lên các nền tảng điện toán đám mây như Microsoft Azure hoặc AWS để hệ thống có thể vận hành chính thức trên Internet.

TÀI LIỆU THAM KHẢO

- Nguồn Internet tham khảo

[Tài liệu tham khảo nghiệp vụ và thiết kế giao diện]

- [1]. PNJ (<https://www.pnj.com.vn>) - Tham khảo quy trình kinh doanh và phân loại danh mục trang sức tại thị trường Việt Nam.
- [2]. Pandora (<https://www.pandora.net>) - Tham khảo UI/UX, phong cách thiết kế giao diện trẻ trung, hiện đại.
- [3]. Tiffany & Co. (<https://www.tiffany.com>) - Tham khảo bố cục hiển thị sản phẩm sang trọng, tối giản.
- [4]. Ruby Stone (<https://rubystone.com.vn>) - Tham khảo thông tin các loại đá quý
- [5]. Ngọc Quý Gemstones (<https://ngocquygemstones.vn>) - Tham khảo cách trình bày nội dung chi tiết sản phẩm ngọc.
- [6]. VNJ Trang Sức Việt (<https://trangsucdaquy.vn>) - Tham khảo cách phân loại bộ lọc sản phẩm.
- [7]. Trang Sức VN (<https://trangsucvn.com>) - Tham khảo nghiệp vụ quản lý giỏ hàng và đặt hàng trực tuyến.

[Tài liệu tham khảo công nghệ lập trình]

- [8]. Microsoft Documentation (<https://learn.microsoft.com/en-us/dotnet/>) - Tài liệu chính thức về nền tảng ASP.NET Core, C# và Entity Framework.
- [9]. W3Schools (<https://www.w3schools.com>) - Tài liệu hướng dẫn sử dụng HTML, CSS, JavaScript và Fetch API.
- [10]. VietQR (<https://vietqr.io>) - Cổng tài liệu API hỗ trợ tích hợp tạo mã QR thanh toán động.

[11]. FontAwesome (<https://fontawesome.com>) - Thư viện biểu tượng sử dụng cho giao diện Front-end.

- Nguồn sách/tài liệu tham khảo

[12]. Christian Nagel (2021), *Professional C# and .NET*. Tác giả cung cấp kiến thức nền tảng về lập trình C# và kiến trúc phần mềm.

[13]. Itzik Ben-Gan (2016), *T-SQL Fundamentals*. Tài liệu tham khảo truy vấn cơ sở dữ liệu SQL Server.